

2025-05

Training-Free Layer Selection for Parameter-Efficient Fine-Tuning of Language Models

Biswas, Aldrin Kabya

IUB

<http://ar.iub.edu.bd/handle/11348/997>

Downloaded from IUB Academic Repository



Training-Free Layer Selection for Parameter-Efficient Fine-Tuning of Language Models

May 2025

Prepared by:

Aldrin Kabya Biswas
ID: 2030443

Department of Computer Science and Engineering
Independent University, Bangladesh

Supervised by

Amin Ahsan Ali
Professor
Department of Computer Science and Engineering
Independent University, Bangladesh

Attestation

I hereby declare that I am fully aware that plagiarism is strictly prohibited and is a violation of the rules and regulations of the university. I affirm that the work presented in this report is my own and reflects original effort. Wherever the work of others has been used, including copyrighted material, ideas, models, data, text, or figures, it has been properly cited and acknowledged in accordance with international academic standards and referencing guidelines.

Author Name:

.....

Signature:

.....

Evaluation Committee

Supervisor

Name: _____ Signature: _____

Internal Examiner 1

Name: _____ Signature: _____

Internal Examiner 2

Name: _____ Signature: _____

External Examiner

Name: _____ Signature: _____

Acknowledgement

I would like to extend my deepest gratitude to my supervisors, Professor Amin Ahsan Ali and AKM Mahbubur Rahman of Independent University, Bangladesh. Their guidance was invaluable, and their insightful suggestions, patience, and generosity with their time were pivotal to bringing this project to fruition. The quality and depth of this work owe much to their expertise and mentorship, and I am truly grateful for their constant support throughout this academic journey.

My sincere thanks also go to the CCDS Lab (Center for Computational & Data Sciences) for their outstanding help and unwavering support. The collaborative spirit and resources within the CCDS Lab greatly facilitated the development and execution of this project. The encouragement and opportunities provided were instrumental in shaping my research path and significantly enriched my learning experience.

Finally, I wish to acknowledge and thank everyone who supported me, directly or indirectly, throughout this difficult yet exciting journey.

Abstract

Adapting large language models (LLMs) to downstream tasks via full fine-tuning is computationally prohibitive. While parameter-efficient fine-tuning (PEFT) methods exist, they often rely on predefined heuristics, incur training overhead for parameter selection, or suffer from poor generalization. This work introduces a novel, training-free layer selection strategy for partial fine-tuning. Our approach leverages the geometric relationships between layer representations by computing the cosine similarity of the [CLS] token embeddings across all layers before fine-tuning begins, using only a single forward pass on a sample of the data. This yields layer-wise importance scores, allowing us to strategically select a small subset of layers for adaptation while freezing the rest. Extensive experiments across 15 diverse NLP tasks, including single-sentence and sentence-pair classifications, demonstrate that our method consistently outperforms various PEFT baselines, including heuristic selections, dynamic/gradient-based methods, and I/O similarity-based selection. Critically, it achieves performance remarkably close (often within 1-2%) to full fine-tuning while drastically reducing trainable parameters by up to 75% and reaching training speedups of $1.5\times$. Furthermore, the method exhibits superior robustness in cross-domain evaluations compared to baselines and generalizes effectively across different model architectures. By exploiting inherent structural properties of pre-trained models via inter-layer [CLS] token similarity, our approach offers an efficient, effective, and robust paradigm for partial LLM fine-tuning.

Contents

1	Introduction	9
1.1	Background of the Work	9
1.2	Problem Definition	10
1.3	Contributions	10
2	Background and Related Work	12
2.1	Preliminaries	12
2.1.1	The Transformer Architecture	12
2.1.2	Pre-training Objectives	13
2.1.3	Fine-tuning Strategies	14
2.2	Existing Partial Fine-tuning Techniques	16
2.2.1	Heuristic Layer Selection and Freezing Strategies	16
2.2.2	Dynamic Layer Selection and Efficient Training	17
2.3	Issues with Existing Methods	18
2.3.1	Issues with Heuristic Layer Selection Strategies	18
2.3.2	Issues with Dynamic Layer Selection Strategies	20
3	Methodology	22
3.1	Motivation	22
3.1.1	Freezing Layers to Preserve Model Architecture	22
3.1.2	Cosine Similarity for Measuring Layer Importance	23
3.1.3	Significance of the [CLS] Token	24
3.1.4	Measuring Inter-Layer Relationships via [CLS] Similarity	24
3.2	k -Layer Selective Fine-tuning	25
3.2.1	Selection Stage	25
3.2.2	Partial Finetuning Stage	28
4	Baselines	31
4.1	Standard Baselines	31
4.1.1	Full Fine-tuning	31
4.1.2	LoRA	32
4.2	Partial Finetuning Baselines	33
4.2.1	Random k Layers	33
4.2.2	Bottom k Layers	34
4.2.3	Top k Layers	35
4.2.4	LIFT	36
4.2.5	RECAP	37
4.2.6	SlimFit	38
4.3	I/O Similarity-based Selection Baseline	39
4.3.1	k Low Similarity (Important)	40

4.3.2	k High Similarity (Redundant)	40
5	Experimental Design	42
5.1	Datasets	42
5.2	Models	44
5.3	Hyperparameters	47
5.4	Evaluation Metrics	48
6	Results and Analysis	50
6.1	Results	50
6.1.1	Single Sentence Tasks	50
6.1.2	Sentence Pair Tasks	52
6.1.3	Training Time and Parameters	54
6.1.4	Layer Selection Insights	54
6.2	Cross-Domain Evaluation	56
6.3	Ablation	58
6.3.1	Performance across Different k Values	58
6.3.2	Performance across Different LLMs	59
7	Conclusion and Future Work	61
7.1	Conclusion	61
7.2	Limitations	62
7.3	Impact on the Society	63
7.3.1	Sustainability of the Work	63
7.3.2	Social and Environmental Effects and Analysis	63
7.3.3	Ethical Considerations	64
7.4	Future Work	64
	Bibliography	64

List of Figures

1	Illustration of Post-Norm vs Pre-Norm Transformer Architecture [12]. . . .	24
2	Overall architecture for k -Layer Selective Fine-tuning. We adopt a two-stage approach: a selection stage and a fine-tuning stage. In the selection stage, we calculate the cosine similarity between the [CLS] token of each layer with all other layers to find layer-wise scores. In the fine-tuning stage, we select the highest/lowest scoring k layers and fine-tune only those while keeping the remaining layers frozen.	26
3	Distribution of layer-wise cosine similarity scores, averaged across all evaluated datasets. During the selection stage, these scores are calculated on a per-sample basis for each dataset so the actual distribution for each sample exhibits greater variance and more pronounced fluctuations than this averaged view.	55
4	Comparison of accuracy and speedup across k for the SST-2 dataset	58
5	Comparison of accuracy and speedup across k for the QNLI dataset	58

List of Tables

5.1	The datasets evaluated in this work. $ \mathcal{Y} $: number of classes for classification tasks (with one exception: STS-B is a real-valued regression task over the interval $[0, 5]$). L : average number of words in input sentence(s) [13].	44
6.1	Results for Single Sentence Tasks. We compare our method with full fine-tuning, LoRA fine-tuning, partial fine-tuning, LIFT, RECAP, SlimFit, and input-output similarity baselines. We use $k=3$ for all layer selection baselines. All the results are the mean of 3 runs with different seeds [0, 42, 100]. Bold indicates the best-performing model and <u>underline</u> indicates the second best.	50
6.2	Results for Sentence Pair Tasks. We compare our method with full fine-tuning, LoRA fine-tuning, partial fine-tuning, LIFT, RECAP, SlimFit, and input-output similarity baselines. We use $k=3$ for all layer selection baselines. All the results are the mean of 3 runs with different seeds [0, 42, 100]. Bold indicates the best-performing model and <u>underline</u> indicates the second best.	52
6.3	Comparison on number of trainable parameters and average speedup. We take the average of the speedup factor of all four of our layer selection methods. We set $k=3$.	54
6.4	Comparison of Cross-Domain Performance. For our method, we show the results for the high scoring k layers, for $k=3$. Bold indicates the best-performing model and <u>underline</u> indicates the second best.	57
6.5	Comparison of performance for different language models. We use $k=3$ for all layer selection methods.	60

1 Introduction

1.1 Background of the Work

Language Models (LMs) have fundamentally transformed the field of Natural Language Processing (NLP) by equipping machines with the ability to interpret and generate human language in a manner that captures both subtle nuances and complex patterns. Pre-trained on vast and diverse corpora, these models develop rich, contextualized word representations that enable them to discern intricate relationships between words and phrases [2]. This capacity allows LMs to excel in a variety of tasks, ranging from text classification and sentiment analysis to more advanced applications like summarization and machine translation [9]. The evolution of LMs has not only led to breakthroughs in performance but has also spurred extensive research into understanding the underlying mechanisms that empower these models [32]. Their ability to grasp semantic, syntactic, and pragmatic aspects of language underscores their potential in driving forward both academic research and real-world applications in NLP.

A key discovery in the exploration of LMs is the hierarchical nature of the representations they learn. Within these models, each layer is responsible for capturing different aspects of linguistic information, ranging from simple word co-occurrence patterns in the lower layers to complex abstractions in the higher layers [23]. This layered approach means that while early layers may focus on basic grammatical structures, later layers tend to encode more refined and abstract semantic features. Such a hierarchical organization not only enhances the model’s overall performance across diverse tasks but also provides valuable insights into how language comprehension is structured within these deep networks. Understanding this layered complexity is critical, as it reveals that not all components of the model contribute equally to every downstream task. This realization has sparked further investigations into how best to leverage the distinct roles of different layers for improved task-specific performance [5, 17].

Despite their impressive capabilities, the sheer scale and complexity of modern LMs pose significant challenges, particularly in terms of computational and memory requirements. Fully fine-tuning these large models for each new task can be prohibitively resource-intensive, creating bottlenecks in both research and deployment. This challenge has led to a growing interest in developing strategies that focus on adapting only a limited subset of the model’s parameters. By concentrating on the most impactful components, these parameter-efficient fine-tuning approaches aim to maintain the robust, pre-learned representations while significantly reducing the computational overhead. This balance between performance and efficiency is critical for making advanced language models more accessible and practical for a wide range of applications, from academic research to industrial implementation.

1.2 Problem Definition

The rapid growth of large-scale pretrained language models has spurred extensive research into parameter-efficient fine-tuning methods. Two major lines of work have emerged in the realm of partial fine-tuning: one that leverages dynamic layer selection based on gradient analysis, selective weight updates, or attention-like mechanisms, and another that relies on fixed heuristic layer freezing strategies. These methods have been designed to reduce the computational overhead associated with fully fine-tuning enormous models while retaining most of their representational power. However, despite their promise, these approaches suffer from several limitations that ultimately hinder their practical utility and robustness, necessitating a closer examination of the underlying challenges.

The first challenge arises from dynamic layer selection-based approaches. These methods require recalculating gradient-based similarity scores or distance metrics for every sample in every training iteration, an operation that not only adds significant computational overhead but also increases memory usage. Moreover, many of these methods rely on auxiliary networks or extra modules to guide the layer selection process, which must be updated concurrently during training. This additional overhead not only prolongs training time but also complicates the optimization process, making it more difficult to achieve consistent convergence. Consequently, while dynamic layer selection can in theory adaptively allocate learning resources to the most relevant layers, the resulting efficiency gains are often offset by the increased training complexity and runtime costs.

A second significant issue stems from predefined heuristic-based strategies that employ random layer freezing, bias-only updates, or layer selection based solely on position or similarity between input and output representations. These methods tend to overlook the complex interdependencies among layers within the pre-trained model, thereby failing to capture the full spectrum of hierarchical representations. By ignoring these inter-layer relations, the fine-tuning process can inadvertently sacrifice important information encoded in the model, resulting in suboptimal performance. Although such strategies simplify the training process by reducing the number of trainable parameters, the trade-off is a loss in performance competitiveness compared to methods that more carefully consider the intricate interactions between layers.

A third critical problem is the pronounced drop in performance observed during cross-domain evaluations. Models fine-tuned using either dynamic layer selection or heuristic-based freezing strategies often exhibit a marked decline in generalization when applied to data from domains different from the training set. This issue is particularly problematic in real-world scenarios, where models are expected to adapt seamlessly to varying contexts and diverse data distributions. The failure to maintain robust performance across domains not only limits the practical applicability of these methods but also highlights an inherent weakness in their ability to preserve the comprehensive knowledge acquired during pre-training. Addressing this challenge is essential for developing fine-tuning strategies that can reliably harness the full potential of large language models across varied and unforeseen environments.

1.3 Contributions

This work highlights a significant leap forward in parameter-efficient fine-tuning techniques for large language models. By using the distinctive [CLS] token representations to compute inter-layer similarities, our approach addresses key limitations in existing methods and provides a new perspective on layer selection for fine-tuning. This work not

only simplifies the selection process by utilizing representations derived during a single forward pass but also enhances the generalization capabilities of the model. The following outcomes encapsulate the core contributions of this work:

- **Comprehensive Analysis of Partial Fine-Tuning Methods:** We conducted an extensive review and categorization of existing partial fine-tuning approaches that select a subset of pre-trained parameters. This study helped us in identifying the strengths and weaknesses of dynamic selection methods and heuristic-based layer selection strategies and laid the groundwork for developing more robust and efficient fine-tuning strategies introduced in this work.
- **Novel Training-Free Layer Selection Approach:** We propose a new method for layer selection that is entirely training-free and based solely on the cosine similarity of [CLS] token representations. By using the [CLS] token, which effectively captures the holistic contextual information of the input, we can compute inter-layer similarities during a single forward pass, leading to a cost-effective and efficient selection process for fine-tuning that explicitly accounts for inter-layer relationships.
- **Extensive Benchmarking Across Diverse NLP Tasks:** Our approach was rigorously evaluated on 15 different NLP tasks, covering 8 single-sentence tasks and 7 sentence-pair tasks. This extensive comparison demonstrates that our method achieves competitive, and in many cases superior, performance compared to existing partial fine-tuning methods, underscoring its effectiveness across a broad range of applications.
- **Robust Cross-Domain Evaluation:** We further validated our approach in a cross-domain setting, comparing its performance against established baselines. The results reveal that our method not only excels within the domain on which it was trained but also generalizes well to other domains, thereby enhancing the overall robustness and adaptability of the fine-tuned models.

These contributions pave the way for a more efficient and effective fine-tuning paradigm for large language models. By introducing an innovative training-free layer selection approach based solely on [CLS] token cosine similarity, which, to the best of our knowledge, is the first of its kind, and validating our method through extensive task-specific and cross-domain evaluations, our work contributes valuable insights and practical solutions to the field of parameter-efficient fine-tuning. These advancements not only reduce computational overhead but also enhance the adaptability and generalization of fine-tuned models, setting a new benchmark for future research in this domain.

2 Background and Related Work

2.1 Preliminaries

2.1.1 The Transformer Architecture

The transformer architecture, introduced by Vaswani et al. [41], is a sequence-to-sequence model that has transformed natural language processing and other sequence modeling tasks. Unlike earlier models that depended on recurrent or convolutional neural networks, the transformer relies entirely on attention mechanisms. This design enables it to effectively capture long-range dependencies within sequences and allows for greater parallelization, enhancing computational efficiency.

Central to the transformer is the self-attention mechanism, which permits the model to assess the significance of various parts of the input sequence when determining the representation of each position. This is implemented through scaled dot-product attention, mathematically expressed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (2.1)$$

Here, Q , K , and V represent the query, key, and value matrices, respectively, and d_k denotes the dimensionality of the key vectors. These matrices are derived as linear projections of the input embeddings using learned weight matrices. The scaling factor $\sqrt{d_k}$ mitigates the risk of large dot product values, which could destabilize the softmax operation, especially in high-dimensional settings.

To enhance its ability to focus on different aspects of the sequence simultaneously, the transformer incorporates multi-head attention. This mechanism applies the attention process multiple times in parallel, each with distinct learned linear projections. The outputs from these attention heads are then concatenated and transformed linearly:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (2.2)$$

where each attention head is defined as:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (2.3)$$

The projection matrices are $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$, and $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$, with h indicating the number of attention heads, d_{model} the model's dimensionality, and d_v the dimensionality of the value vectors.

The transformer architecture comprises two main components: an encoder and a decoder, each structured as a stack of identical layers. An encoder layer consists of two sub-layers: a multi-head self-attention mechanism and a position-wise feed-forward network. Each sub-layer is followed by a residual connection and layer normalization, formulated as:

$$\text{LayerNorm}(x + \text{Sublayer}(x)) \quad (2.4)$$

where $\text{Sublayer}(x)$ is the output of the respective sub-layer. The position-wise feed-forward network applies identical linear transformations to each position independently:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (2.5)$$

where W_1 , W_2 , b_1 , and b_2 are learned parameters, and the $\max(0, \cdot)$ introduces non-linearity via the ReLU activation function.

The decoder mirrors the encoder’s stacked structure but includes an additional sub-layer, resulting in three sub-layers per layer: a masked multi-head self-attention mechanism, a multi-head attention mechanism that attends to the encoder’s output, and a position-wise feed-forward network. The masking in the self-attention sub-layer ensures that the model only attends to earlier positions in the sequence, preventing it from accessing future tokens during training. This is critical for autoregressive tasks like language generation. Similar to the encoder, each sub-layer in the decoder is followed by residual connections and layer normalization.

Unlike recurrent networks, the transformer lacks an inherent sense of sequence order due to its non-sequential processing. To address this, positional encodings are added to the input embeddings, providing positional information to the model. These encodings are generated using sine and cosine functions:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (2.6)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (2.7)$$

where pos is the position in the sequence, and i is the dimension index. These encodings have the same dimensionality as the input embeddings d_{model} and are added element-wise to the embeddings before processing by the encoder and decoder.

2.1.2 Pre-training Objectives

The remarkable capabilities of modern transformer-based language models stem significantly from the pre-training phase, where the model learns general-purpose linguistic representations from vast amounts of unlabeled text data. This process typically involves optimizing the model based on one or more self-supervised objectives, forcing it to understand statistical patterns, syntax, semantics, and even commonsense knowledge embedded within the text corpus.

A predominant objective, particularly for models like BERT and its variants, is Masked Language Modeling (MLM). During MLM, a certain percentage of input tokens in a sequence are randomly masked (e.g., replaced with a special ‘[MASK]’ token), and the model’s task is to predict the original identity of these masked tokens based solely on the surrounding unmasked context. Because the transformer’s self-attention mechanism can attend to tokens both to the left and right, MLM enables the training of deep bidirectional representations, allowing the model to build a richer understanding of context compared to unidirectional models.

Another objective employed in the original BERT pre-training was Next Sentence Prediction (NSP). For this task, the model receives two text segments, A and B, and must predict whether segment B is the actual sentence that follows segment A in the original corpus or just a random sentence drawn from the corpus. The goal of NSP was

to help the model understand sentence relationships, which is beneficial for downstream tasks like Natural Language Inference (NLI). However, subsequent research indicated that the NSP objective might be less impactful than initially thought, and potentially even detrimental in some cases, leading models like RoBERTa to abandon it in favor of focusing solely on MLM or variations thereof, often with dynamic masking where the mask pattern changes across training epochs.

Contrastingly, autoregressive language models, such as the GPT family, are typically pre-trained using a Causal Language Modeling (CLM) objective, often referred to as next-token prediction. In this approach, the model processes the input sequence unidirectionally (usually left-to-right) and is trained to predict the next token at each position, given all the preceding tokens. This objective naturally lends itself to text generation tasks.

Regardless of the specific objective(s) used, the outcome of the pre-training phase is a model with weights that encode a broad understanding of language structure and meaning. These pre-trained weights serve as a powerful starting point for adaptation to various specific downstream NLP tasks, a process known as fine-tuning.

2.1.3 Fine-tuning Strategies

Full Fine-tuning

Full fine-tuning represents the traditional and often most straightforward approach to adapting a pre-trained language model (PLM). In this strategy, the entire set of parameters within the pre-trained model, along with the parameters of the newly added task-specific head, are updated during the training process on the downstream task data. The underlying principle is to allow the model complete flexibility to adjust its learned representations, from the initial input embeddings through every layer of the Transformer stack, to best suit the subtleties and demands of the specific target task.

The process typically begins by initializing the task-specific head with random weights, while the main body of the model retains its pre-trained weights. Training then proceeds using a standard optimization algorithm, like AdamW, minimizing a task-specific loss function (e.g., cross-entropy for classification). Gradients are computed with respect to all model parameters, both the head and the entire pre-trained backbone, and used to update the weights. This allows for potentially deep modifications to the model’s internal workings, enabling it to capture complex task-specific patterns.

The primary advantage of full fine-tuning lies in its potential to achieve the highest possible performance on the target task. By allowing all parameters to adapt, the model can theoretically find the optimal configuration within its parameter space for the new data distribution and objective. It fully leverages the capacity of the large pre-trained model, making no assumptions about which parts might be more or less important for the new task.

However, this comprehensive adaptation comes at a significant cost, particularly as PLMs continue to grow in size. Updating all parameters, often numbering in the hundreds of millions or billions, is computationally expensive. It requires substantial GPU memory to store the model weights, activations during the forward pass, gradients during the backward pass, and optimizer states (which can often double or triple the memory required for the weights alone, depending on the optimizer). Training time is also considerably longer compared to methods that update fewer parameters. Furthermore, full fine-tuning carries a higher risk of overfitting, especially when the downstream task dataset is small or medium-sized. The model’s vast number of parameters can easily memorize the training data rather than learning generalizable patterns. Another potential issue is “catastrophic

forgetting,” where the model’s extensive adjustments for the new task might degrade the general linguistic knowledge acquired during pre-training, harming its performance on related tasks or its overall robustness. These substantial resource requirements and potential drawbacks have motivated the development of alternative, more efficient fine-tuning strategies.

Parameter-Efficient Fine-tuning (PEFT)

In response to the significant computational demands and potential drawbacks of full fine-tuning, a diverse family of techniques known as Parameter-Efficient Fine-tuning (PEFT) has emerged. The core objective of PEFT methods is to adapt large pre-trained language models to downstream tasks by modifying only a small fraction of the model’s parameters or by introducing a minimal number of new, trainable parameters, while keeping the bulk of the original pre-trained model frozen. This approach aims to significantly reduce the computational overhead (memory, storage, training time) associated with fine-tuning, making it feasible to adapt large models even in resource-constrained settings, while striving to maintain performance levels close to those achieved by full fine-tuning.

Several distinct strategies fall under the PEFT framework, each operating on different principles. One prominent category involves adding small, trainable modules into the pre-trained architecture. Adapter modules [18], for instance, are typically small feed-forward networks inserted between the layers of a frozen Transformer. Only the parameters of these adapters are trained, allowing the model to learn task-specific functions without altering the original weights. While highly parameter-efficient, adapters introduce extra computational steps during inference, leading to increased latency, and require modifications to the base model architecture.

Another widely adopted PEFT technique is Low-Rank Adaptation (LoRA) [19]. Instead of adding separate modules, LoRA hypothesizes that the change in weights required for task adaptation has a low “intrinsic rank.” It injects trainable, low-rank decomposition matrices into existing weight matrices (commonly the query and value projection matrices in self-attention layers) of the frozen pre-trained model. During training, only these small low-rank matrices are updated. LoRA is advantageous as it doesn’t introduce inference latency (the low-rank updates can be merged with the original weights after training) and requires no architectural changes. However, its effectiveness might depend on the validity of the low-rank hypothesis for a given task, and it still necessitates computing gradients through the entire frozen backbone during the backward pass, even if only the LoRA parameters are updated.

A different approach focuses on manipulating the input or activations rather than the model weights directly. Prompt Tuning [25] prepends trainable “soft prompt” vectors to the input sequence embeddings, keeping the entire PLM frozen. The model learns to condition its behavior on these learned prompt vectors to perform the downstream task. Prefix Tuning [26] is similar but adds trainable prefix vectors to the hidden states in each Transformer layer. These methods are extremely parameter-efficient, as only the prompt or prefix vectors are trained, making it easy to store and switch adaptations for many tasks. However, they can sometimes be challenging to optimize and may not reach the same peak performance as methods that modify model weights, especially on more complex tasks, as they lack direct control over the model’s internal computations.

The focus of this work, Partial or Selective Fine-tuning, represents yet another distinct PEFT strategy. Unlike adapters or prompt/prefix tuning, it does not introduce new parameters. Instead, it involves selecting and updating only a specific subset of the original pre-trained model’s parameters, while freezing all others. Examples include fine-

tuning only the bias terms (such as in BitFit [48]), updating only the final few layers, or, as explored extensively in this work, selecting specific layers based on certain criteria. This approach differs fundamentally from LoRA because it directly modifies a subset of the original weights, rather than approximating the update with a low-rank matrix.

The investigation into partial fine-tuning, specifically layer-wise selection, is motivated by several factors that distinguish it from other PEFT methods and address some of their limitations. Firstly, it directly leverages the existing, richly parameterized structure of the pre-trained model, hypothesizing that targeted updates to carefully chosen original components might be more effective than adding external modules or relying solely on input manipulation. Secondly, it avoids the potential inference latency introduced by adapter modules and the architectural modifications they require. Thirdly, compared to prompt-based methods, it offers a more direct way to influence the model’s internal representations and computational pathways, which could be crucial for complex reasoning tasks.

While LoRA is effective, partial fine-tuning explores an alternative hypothesis: that adaptation might not always be low-rank, and directly fine-tuning a strategically chosen subset of full-rank parameters might yield better results or different trade-offs. Furthermore, freezing a large portion of the original network (often significantly more than just keeping the backbone frozen for gradient calculation as in LoRA or Adapters) can lead to substantial savings in optimizer state memory and reduced computational load during the backward pass, depending on implementation. By focusing on identifying which existing parts of the model are most crucial for adaptation, partial fine-tuning aims to achieve efficiency while preserving more of the model’s inherent structure and pre-trained knowledge compared to methods that rely heavily on newly added parameters or low-rank approximations. This exploration seeks a balance between efficiency and the ability to make precise, impactful adjustments within the original model architecture.

2.2 Existing Partial Fine-tuning Techniques

Numerous partial fine-tuning strategies have been introduced in several works that involves updating only a selected subset of the model’s original parameters. These can be divided broadly into two categories: Heuristic Layer Selection and Dynamic Layer Selection. Below, we provide a literature review of all the existing partial fine-tuning techniques under these categories.

2.2.1 Heuristic Layer Selection and Freezing Strategies

- Lee et al. (2019) [24] investigated the effectiveness of freezing different numbers of final layers in pre-trained Transformer models (specifically BERT and RoBERTa) during fine-tuning. Across a variety of NLU tasks, they found that fine-tuning only the last 25% of layers can achieve 90% of the full model’s performance, suggesting redundancy in updating earlier layers. Interestingly, they also observed that, for certain tasks, freezing some layers resulted in better performance than fine-tuning the entire model, indicating potential overparameterization.
- Ben-Zaken et al. (2021) [48] introduced BitFit, a parameter-efficient fine-tuning technique for pre-trained language models that focuses on updating only the bias terms (or a subset of them) while freezing all other parameters. They demonstrate that, especially for small to medium datasets, BitFit can achieve performance comparable to full fine-tuning, and for larger datasets remains competitive with other

sparse fine-tuning methods. Their work suggests that much of the benefit of fine-tuning comes from accessing pre-trained knowledge rather than learning new task-specific information.

- Shen et al. (2021) [36] argued against simply transferring all learned knowledge from base classes to novel classes in few-shot learning, proposing instead a “partial transfer” approach. They used an evolutionary search to determine which layers of a pre-trained model on base classes should be frozen and which should be fine-tuned with varying learning rates when adapting to novel classes. This search aimed to find an optimal balance between leveraging pre-trained knowledge and adapting to the new, few-shot data.
- Ingle et al. (2022) [22] investigated the impact of freezing different percentages of Transformer layers in RoBERTa when fine-tuning in few-shot settings. Motivated by prior work showing similarity between pre-trained and fine-tuned representations in early layers, they explore freezing these initial layers and find that freezing the first 50% can improve performance and significantly reduce training time compared to training all layers. They also demonstrate the generalizability of this layer freezing approach to other few-shot learning techniques like DNNC [50] and LM-BFF [13].
- Simoulin et al. (2023) [37] introduced a memory-efficient selective fine-tuning approach for Transformer models that, during the backward pass, only propagates gradients through a small, randomly selected subset of input token positions. This allows the model to discard the activations corresponding to the unselected positions during the forward pass, thereby reducing memory usage without modifying the model architecture. They demonstrate that this approach achieves performance comparable to full fine-tuning while requiring significantly less GPU memory, particularly benefiting models with hundreds of millions of parameters.
- He et al. (2024) [16] investigated redundancy in Transformer blocks, MLP layers, and attention layers using the cosine similarity between their input and output, finding surprisingly high redundancy in attention layers, which can be pruned with minimal performance impact. They explored dropping entire blocks, individual MLP layers, individual attention layers, and a combination of both, concluding that attention dropping offers the best efficiency gains and that jointly dropping attention and MLP layers improves performance at higher sparsity levels. Their work also examines the redundancy of these layers throughout training, finding it to be a consistent characteristic, especially for attention.

2.2.2 Dynamic Layer Selection and Efficient Training

- Vucetic et al. (2022) [44] proposed Freeze And Reconfigure (FAR), a memory-efficient fine-tuning method for BERT-like models on resource-constrained devices. FAR freezes a subset of the model’s weights, specifically feed-forward network nodes, based on their change in value during a short initial fine-tuning phase called “priming”, and then dynamically reconfigures the model architecture to group frozen and trainable weights. This reduces the memory footprint of activations and gradients during subsequent fine-tuning.
- Zhu et al. (2023) [52] introduced LIFT, a layer-wise fine-tuning approach for large language models that updates only one Transformer layer per iteration, significantly

reducing both memory usage and training throughput. Unlike existing parameter-efficient fine-tuning methods that still require back-propagation through all layers, LIFT reduces the back-propagation depth by half on average. The proposed method fine-tunes a specific and single block on each iteration, freezing the others and stopping the backpropagation before the frozen blocks. The authors also show that LIFT can be combined with existing PEFT techniques like LoRA for further efficiency gains.

- Pan et al. (2024) [33] observed a skewed distribution of weight norms across layers when fine-tuning LLMs with LoRA, with the bottom and top layers exhibiting significantly larger norms. Based on this, they propose LISA, which uses a form of layerwise importance sampling where only a small subset of the middle layers are updated during each training iteration, effectively freezing the others. This sampling strategy is designed to mimic the observed LoRA behavior and is shown to sometimes outperform both LoRA and full fine-tuning in terms of performance while using a comparable amount of memory.
- Gu et al. (2024) [14] proposed a layer freezing method for efficient fine-tuning, focusing on where to fine-tune based on a semantic analysis. They define the concept of “semantic transition” within a language model’s layers and use the deviation from an idealized transition path to estimate the importance of fine-tuning each layer. This allows them to freeze less important layers (deeper layers in their analysis), reducing computational cost without significant performance degradation. They also introduce different budget allocation and infilling strategies to control the trade-off between performance and efficiency.
- Ilhan et al. (2024) [21] presented RECAP, a framework for resource-efficient pruning of large Transformer models during fine-tuning. RECAP iteratively prunes and fine-tunes the model, using Taylor approximation for importance estimation during pruning and the Fisher information criterion for selecting a subset of weights to update. This process aims to reduce GPU memory usage without sacrificing performance, in contrast to traditional pruning methods that typically require a full fine-tuning step before pruning.
- Ardakani et al. (2024) [1] proposed SlimFit, a memory-efficient fine-tuning approach for transformer-based models that dynamically analyzes training dynamics to freeze less contributory layers. SlimFit combines an inter-layer scheduling (ILS) algorithm with quantization and pruning of specific layers to significantly reduce activation memory usage. The ILS algorithm dynamically ranks layers based on the magnitude of their weight updates and iteratively freezes those with minimal contributions, while quantization and pruning even out activation numbers across layers and minimize the memory footprint of static activations. The method is evaluated in fine-tuning large transformer-based models such as ViT and BERT. The proposed method significantly reduces the memory footprint during fine-tuning.

2.3 Issues with Existing Methods

2.3.1 Issues with Heuristic Layer Selection Strategies

- In Lee et al. (2019) [24], the primary limitation lies in its methodological approach to layer selection for fine-tuning. The authors’ core investigation revolves around

the question of how many of the final layers of pretrained transformer models need to be fine-tuned to achieve optimal performance on downstream tasks. While comprehensive in its evaluation across multiple datasets, the study’s approach is fundamentally heuristic. It explores freezing a progressively increasing number of initial layers, observing the resulting change in task-specific effectiveness. This method, while providing empirical evidence for the redundancy of some initial layers, lacks a principled mechanism for identifying which specific layers are most crucial for a given task. The selection is based solely on layer position (proximity to the output), without considering the learned representations or inter-layer dependencies within the model. This can lead to suboptimal fine-tuning strategies, particularly when the task requires specific information encoded in earlier layers.

- Ben-Zaken et al. (2021) [48] proposed fine-tuning only the bias terms (or a subset thereof) of a pre-trained transformer. While empirically effective, BitFit’s main limitation is its uniform application of the bias-only constraint across all layers and tasks. It assumes that bias terms are universally the most important parameters for adaptation, irrespective of the specific downstream task. This contrasts with the understanding that different layers in a transformer capture different levels of linguistic abstraction. Certain tasks might rely more heavily on information encoded in specific layers (e.g., syntactic information in earlier layers, semantic information in later layers). By universally restricting fine-tuning to bias terms, BitFit might prevent necessary adjustments in layers that are crucial for a given task, hindering optimal performance. The method does not consider the heterogeneity in the type of features and their importance for the downstream tasks.
- Shen et al. (2021) [36] addressed few-shot classification by proposing a method to selectively fine-tune layers of a pre-trained model, using an evolutionary search to determine which layers to freeze or fine-tune, and with what learning rate. While the paper introduces a valuable layer-wise selection, its primary limitation lies in its reliance on an evolutionary search algorithm. Evolutionary algorithms, while powerful, are computationally expensive. The search process itself requires numerous forward and backward passes through the network for each candidate configuration, evaluating performance on a validation set. This significantly increases the computational cost compared to full fine-tuning or simpler partial fine-tuning approaches. Furthermore, evolutionary search, as a meta-heuristic, doesn’t guarantee finding the globally optimal solution, potentially settling for a locally optimal layer selection strategy, especially in case of the high complexity of the task.
- Ingle et al. (2022) [22] explores the impact of freezing initial Transformer layers in RoBERTa for few-shot text classification. The paper’s primary limitation lies in its heuristic and fixed approach to layer selection. The authors investigate freezing a predetermined percentage (initially 50%, then varying in coarse steps of 25%) of the initial layers, without a data-driven or task-specific mechanism for determining which layers are truly optimal to freeze, also does not incorporate any calculations to decide what layers to freeze or not, rather it is experimentally chosen. This contrasts with a more adaptive approach that might identify crucial layers based on their learned representations or contribution to the specific few-shot task. The analysis is largely empirical, relying on observing performance changes across different freezing configurations, and is also limited in scope to a few-shot setting.
- Simoulin et al. (2023) [37] proposed a method to reduce GPU memory usage during fine-tuning by selectively propagating gradients through only a subset of input

positions. While this approach is effective in reducing memory consumption, it has a key limitation: the selection of input positions is random. The paper does not propose a method for picking, based on a principled approach, the best tokens for a task, and focuses only on randomly picking k positions. This randomness means that the method does not take into account the varying importance or relevance of different parts of the input sequence to the downstream task. Crucial information for a given task might reside in the non-selected (frozen) positions, potentially leading to performance degradation.

- He et al. (2024) [16] investigates redundancy within Transformer components, particularly focusing on Attention and MLP layers. The paper primarily uses a post-hoc analysis, dropping layers after full training, to assess their importance based on the cosine similarity between layer inputs and outputs. While this approach reveals inherent redundancies, it does not directly translate to efficient fine-tuning. The study identifies that many attention layers can be removed without significantly impacting performance, but it doesn’t provide a mechanism for selectively fine-tuning only the crucial layers during the adaptation to a new task. The paper thus only focuses on identifying the redundancy after the full training, rather than a training-free mechanism to do layer selection.

2.3.2 Issues with Dynamic Layer Selection Strategies

- Vucetic et al. (2022) [44] introduced Freeze and Reconfigure (FAR) to reduce memory usage during BERT fine-tuning on resource-constrained devices. FAR’s primary limitation lies in its reliance on L1 norm of weight updates during the initial fine-tuning or “priming” phase, which adds computational overhead, followed by a structured selection strategy through grouping of contiguous nodes within FFN layers. This approach, while promoting memory efficiency through structured freezing and reconfiguration, requires an initial training stage, and might not capture the optimal set of parameters for fine-tuning on diverse downstream tasks. The focus on contiguous groups, dictated by memory access efficiency, restricts the selection flexibility, potentially excluding important individual parameters or more complex inter-layer dependencies crucial for optimal performance.
- Zhu et al. (2023) [52] proposed LIFT that updates only one layer of a transformer model at a time, sequentially, to improve memory and computational efficiency. While LIFT achieves its stated goals of reducing memory usage and increasing throughput, the core approach is still a training-based method, which, does require a full forward and backward pass. It cycles through updating each layer individually. This sequential, layer-by-layer update strategy differs fundamentally and significantly from selecting specific layers beforehand in a training-free manner. Although LIFT reduces back-propagation depth on average, it still requires training iterations and does not provide a mechanism for selecting layers without training.
- Pan et al. (2024) [33] introduced LISA that selectively updates layers during fine-tuning based on an importance sampling strategy derived from weight norms. While LISA aims for memory efficiency and improved performance, a core issue is that its layer selection fundamentally relies on a training-dependent metric (weight norms). LISA utilizes training checkpoints to get the weights and only then applies importance sampling, which means that the whole method is not training-free, but depends on some initial training.

- Gu et al. (2024) [14] proposed a method that leverages semantic analysis to determine which layers to freeze during fine-tuning. The approach uses a virtual transition of the latent representation and its factual transition to identify deviations. It has the following issues. First, although it reduces backpropagation costs, it still uses fine-tuning to find the best layers, and uses multiple iterations over the data and different training settings, so it needs additional training, and does not involve an up-front, training-free layer selection. Second, its transition route is not general for all tasks, it is specifically designed for next-token prediction and it is not trivial to extend it to other tasks.
- Ilhan et al. (2024) [21] presented RECAP that relies on pruning transformer-based models during fine-tuning to improve efficiency. RECAP’s approach involves an iterative process of pruning, fine-tuning, and updating, guided by Taylor-approximation for importance estimation and Fisher information for weight updates. RECAP needs multiple iterations of fine-tuning. This iterative nature introduces a significant computational overhead. Although RECAP aims to be resource-efficient, the core methodology still necessitates training to identify which parts of the model to prune and update. It relies on importance scores derived from the model’s behavior during the fine-tuning process and hence is not training-free.
- Ardakani et al. (2024) [1] introduced SlimFit to reduce memory usage during fine-tuning by dynamically freezing layers based on their training dynamics. The method freezes layers with fewer updates, and also contains a stage to select what layers to be frozen. However, the main issue of SlimFit is that its layer selection mechanism is not entirely training-free. The determination of which layers to freeze is based on the training dynamics, specifically, the gradient contributions observed during an initial training phase (iterative freezing). This means that a portion of training must occur before any layers can be frozen, incurring computational and memory overhead. This dynamic, training-dependent freezing process thus adds complexity to the process.

While these studies have significantly contributed to the field of parameter-efficient fine-tuning, they are often hampered by reliance on predefined heuristics or training-dependent procedures that incur significant computational overhead. In contrast, our approach overcomes these limitations by adopting a completely training-free strategy based on the cosine similarity of [CLS] token representations that explicitly accounts for the non-trivial relationships between layers without relying on any predefined heuristics, thereby eliminating the need for costly iterative training and providing a more efficient, cost-effective solution for the fine-tuning of language models.

3 Methodology

3.1 Motivation

In the following sections, we explain the motivation behind the design choices of our fine-tuning method, highlighting the effectiveness of layer freezing, use of cosine similarity and [CLS] tokens, and explain how the cosine similarity between [CLS] tokens effectively captures inter-layer relationships.

3.1.1 Freezing Layers to Preserve Model Architecture

In addition to traditional PEFT techniques outlined in the previous chapter, there is a growing trend towards model compression for large language models (LLMs) that involves pruning the architecture itself, usually targeting dense matrices [3], attention heads [31, 43], filters [30, 34], parameters [46, 40, 20], or entire functional units like layers [7]. Parameter-level pruning, while capable of reducing model size, often induces unstructured sparsity or dimensional incongruities. These structural irregularities violate the assumptions underlying optimized computational kernels and data formats (e.g., dense matrix multiplication routines, compressed sparse representations), making it difficult to execute efficiently on hardware accelerators optimized for dense, regular computations. Layer pruning, on the other hand, maintains structural regularity by removing complete layers, but it fundamentally alters the model’s functional depth and computational graph, risking the loss of critical intermediate representations and necessitating complex retraining or heuristics to identify dispensable layers without catastrophic performance degradation.

In contrast, layer freezing avoids these issues by preserving the entire network architecture and its parameters within the existing structure, only rendering specific layers immutable during subsequent training phases. This preserves the original topological consistency and computational graph structure, resulting in seamless deployment compatibility with highly optimized inference engines and hardware routines while maintaining the full expressive capacity learned during pre-training.

Another significant reason for layer freezing comes from its advantages during model adaptation, such as fine-tuning. Unlike the pruning strategies focused on reducing inference cost, freezing targets the training process itself. By designating specific layers as non-trainable, the computation and storage of gradients associated with the parameters within these layers are entirely bypassed during backpropagation. This significantly reduces the dimensionality of the parameter space over which optimization occurs, drastically lowering the memory footprint required for gradient accumulation and optimizer states (e.g., momentum buffers in Adam). Consequently, the computational cost per training step is substantially diminished, enabling the adaptation of extremely large models on systems with constrained computational or memory resources. This reduction in the effective number of trainable parameters can also simplify the optimization landscape encountered during fine-tuning, accelerating convergence towards task-specific solutions

by focusing learning on a smaller subset of the model’s components.

Furthermore, freezing layers provides a principled mechanism for knowledge preservation and transfer. While layer pruning irrevocably removes the computational function and learned representations of the pruned layers, freezing retains the entirety of the pre-trained knowledge encoded within the layer parameters, allowing them to serve as fixed feature extractors. This explicit preservation can mitigate catastrophic forgetting by stabilizing the representations generated by the frozen portions of the network. From a transfer learning perspective, freezing irrelevant layers allows leveraging powerful, general-purpose representations learned during large-scale pre-training while adapting only important, more task-specific layers. This structural immutability ensures the model retains its full architectural integrity, allowing straightforward integration with complementary optimization techniques like quantization or low-rank adaptation applied to the remaining trainable layers, hence compounding efficiency gains for practical LLM deployment and adaptation.

3.1.2 Cosine Similarity for Measuring Layer Importance

In transformer-based language models employing the Pre-Norm architecture, the magnitude of hidden states tends to increase with depth due to the specific ordering of normalization and residual connections [47]. Unlike Post-Norm architectures, where layer normalization follows the self-attention and feed-forward sub-layers, Pre-Norm applies normalization *before* these sub-layers (See Figure 1). For a given layer $\mathbf{l}$, the hidden state updates as $\mathbf{h}_l = \mathbf{h}_{l-1} + \text{sublayer}(\text{LayerNorm}(\mathbf{h}_{l-1}))$, where sublayer denotes either the attention or feed-forward component. The layer normalization ensures that the sublayer input has zero mean and unit variance, but the residual connection adds the sublayer output directly to the previous hidden state. If the sublayer output aligns positively with $\mathbf{h}_{l-1}$, the norm of $\mathbf{h}_l$ increases. This effect compounds across layers, leading to a systematic growth in hidden state magnitudes, a phenomenon well-documented in empirical studies [28]. This depth-dependent scaling distinguishes Pre-Norm from Post-Norm designs, where normalization constrains magnitudes after each sub-layer.

Given this architectural trait, selecting an appropriate similarity metric for analyzing inter-layer relationships is critical. Magnitude-sensitive metrics like the dot product and Euclidean distance are skewed by the increasing norms of hidden states in deeper layers. The dot product, defined as $\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta$, combines the angle θ between vectors with their magnitudes, causing deeper layers with larger norms to appear more similar even if their directional alignment, indicative of functional similarity, is poor. Similarly, the Euclidean distance, $\|\mathbf{a} - \mathbf{b}\| = \sqrt{\|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2\mathbf{a} \cdot \mathbf{b}}$, decreases for smaller-magnitude vectors (e.g., in earlier layers) and grows with magnitude differences, even when vectors are perfectly aligned directionally. For example, if $\mathbf{b} = k\mathbf{a}$, the distance becomes $|1 - k| \|\mathbf{a}\|$, which scales with magnitude despite $\cos \theta = 1$, masking true functional relationships.

To address these biases, cosine similarity stands out as the optimal metric for assessing layer importance in Pre-Norm transformers. Defined as $\frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|} = \cos \theta$, it measures only the cosine of the angle θ between vectors, rendering it invariant to their magnitudes. This property is invaluable in Pre-Norm architectures, where hidden state norms grow with depth as mentioned. By focusing exclusively on directional alignment, cosine similarity identifies layers with similar representational content, unaffected by scaling artifacts. This makes it ideal for detecting layer redundancies or functional equivalence, aligning with its established use in representation learning and NLP tasks [35, 6]. Thus, cosine similarity provides a robust, unbiased tool for analyzing transformer layer dynamics.

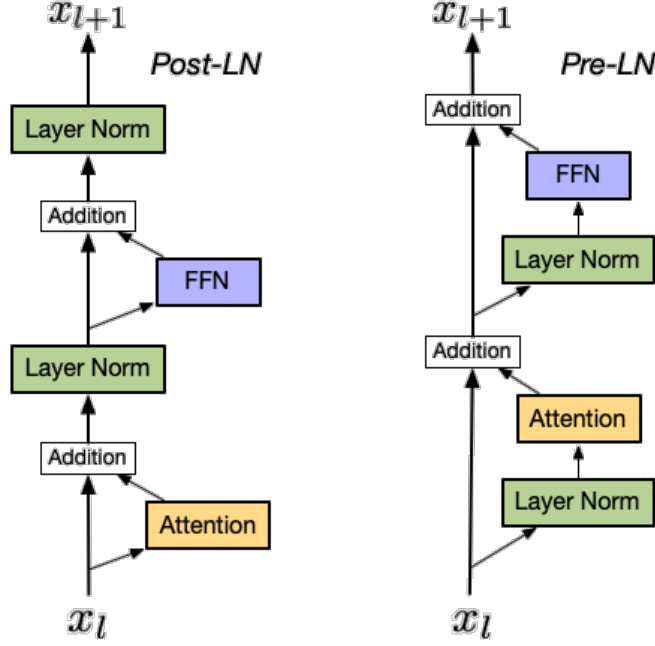


Figure 1: Illustration of Post-Norm vs Pre-Norm Transformer Architecture [12].

3.1.3 Significance of the [CLS] Token

While some studies employ cosine similarity between the input and output hidden states of each layer to identify redundancy in transformer-based language models [16], this approach may not be the most effective for tasks that primarily depend on the aggregate sequence representation. Comparing the entire hidden states involves analyzing high-dimensional representations for every token in the sequence, which captures how much the layer transforms the full set of token embeddings. However, this method can be overly coarse, as changes in individual token representations may not necessarily reflect their contribution to the final task, especially when the task relies on a summarized understanding of the input.

In addition, computing similarity across all hidden states is computationally intensive, particularly for longer sequences, and may introduce noise from token-level variations that are irrelevant to downstream objectives. This makes it less aligned with parameter-efficient fine-tuning strategies that aim to optimize performance while minimizing resource demands.

This highlights the significance of using [CLS] tokens. In models like BERT, the [CLS] token is a special token prepended to the input sequence, designed to encapsulate the overall semantics of the sequence, particularly for classification tasks [11]. By focusing on the [CLS] token, we can directly evaluate how each layer refines this aggregate representation, which is often the primary input to downstream task predictors. A high similarity between the [CLS] tokens of two layers suggests that the intervening layers contribute minimally to enhancing this representation, indicating potential redundancy [8]. This method is computationally efficient, as it reduces the analysis to a single vector per layer, and aligns closely with the fine-tuning objectives of many tasks that utilize the [CLS] token, ensuring that the identified redundant layers are less critical for performance.

3.1.4 Measuring Inter-Layer Relationships via [CLS] Similarity

In language models employing the transformer architecture, the propagation of information across layers is not a strictly linear or sequential process; rather, it is shaped by

mechanisms such as residual connections and self-attention, which enable complex, non-linear dependencies between layers, including those that are not adjacent.

To effectively capture these inter-layer relationships, we can calculate the cosine similarity between the [CLS] tokens of each layer and those of all other layers, rather than restricting the analysis to consecutive pairs. Focusing solely on consecutive layers would limit our understanding to immediate transitions in representation, risking overlooking significant non-local dependencies or representation recurrences, such as when a layer’s features align closely with those of a much deeper or shallower layer. This broader similarity assessment provides a more complete picture of how each layer’s aggregate representation relates to the entire model’s representation space, revealing patterns of consistency or divergence that span the architecture. By considering all pairwise similarities, we ensure that the inter-layer dynamics are evaluated holistically, which is critical for understanding the model’s computational flow and representation evolution.

Averaging these cosine similarities for each layer can give us a single, interpretable score that encapsulates its overall alignment with the representations of all other layers, functioning as a centrality-like measure within the model’s representation network. This average similarity reflects how typical or distinctive a layer’s contribution is: layers with high average similarity may represent core, shared features that are consistent across the model, while those with low average similarity could indicate unique, task-specific information that differentiates them from the broader representation trends.

In the context of parameter-efficient fine-tuning, this measure is theoretically valuable because it enables the strategic selection of layers based on their relational roles within the model. Fine-tuning layers with high average similarity might reinforce general, stable features that underpin the model’s foundational knowledge, whereas adjusting layers with low average similarity could target specialized adaptations crucial for the specific task. Thus, this approach leverages the interconnected nature of transformer layers to inform which parts of the model are most impactful to adjust, optimizing resource use while preserving performance.

3.2 k -Layer Selective Fine-tuning

Our proposed methodology for k -layer selective fine-tuning consists of two stages: (i) the Selection Stage and (ii) the Partial Fine-tuning Stage. In the selection stage, we determine which layers should be fine-tuned, while in the fine-tuning stage, we fine-tune the selected layers along with the classifier head, keeping the remaining layers frozen. The selection stage does not require any training. Details of these stages are provided in Section 3.2.1 and Section 3.2.2 below. The overall model architecture is depicted in Fig. 2.

3.2.1 Selection Stage

The core idea behind our approach is to strategically select a subset of layers from a pre-trained language model for fine-tuning, aiming to optimize performance on downstream tasks while reducing computational overhead and preserving valuable pre-trained knowledge. This selection stage is crucial for identifying which layers are most suitable for adaptation based on their representational characteristics within the context of the entire model hierarchy.

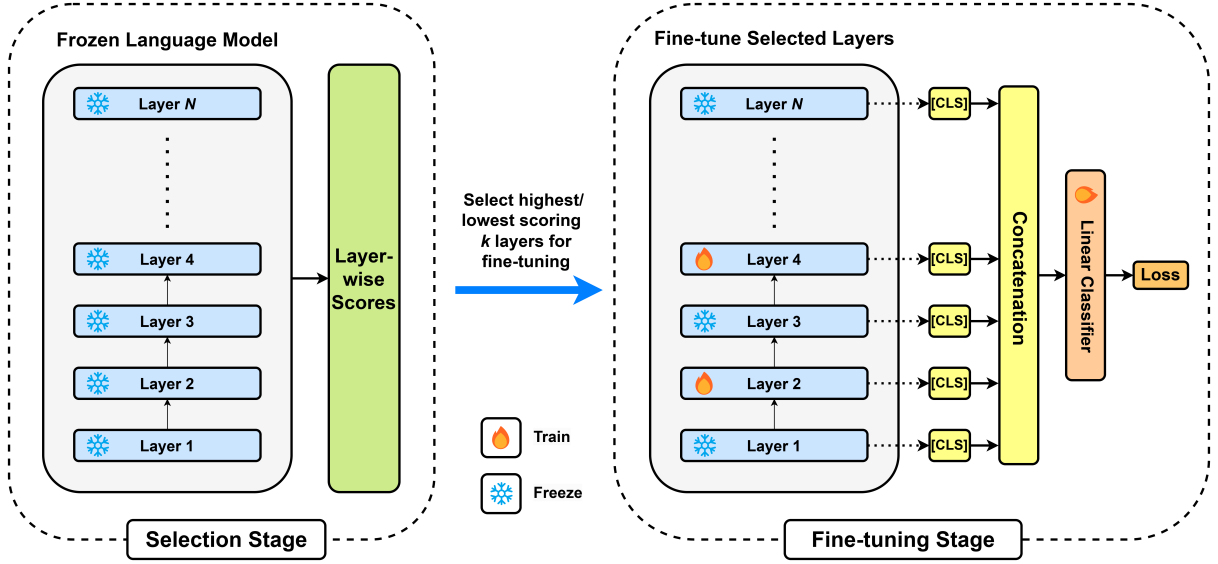


Figure 2: Overall architecture for k -Layer Selective Fine-tuning. We adopt a two-stage approach: a selection stage and a fine-tuning stage. In the selection stage, we calculate the cosine similarity between the [CLS] token of each layer with all other layers to find layer-wise scores. In the fine-tuning stage, we select the highest/lowest scoring k layers and fine-tune only those while keeping the remaining layers frozen.

Step 1: Representation Extraction from the Pre-trained LM

Given an input sentence S , we first process it using a pre-trained language model, denoted as f_{LM} . These models, typically based on the Transformer architecture (e.g., BERT, RoBERTa), consist of multiple layers stacked hierarchically. The input sentence S is initially tokenized into a sequence of subword units. For models like BERT, the special classification token, [CLS], is prepended to the sequence. The tokenized sequence is then fed into f_{LM} .

We extract the contextualized representations generated by the model at each of its layers. Let the total number of layers in f_{LM} be L . For a sentence tokenized into n tokens plus the [CLS] token, the token set can be represented as $\mathcal{T} = \{t_{[CLS]}, t_1, t_2, \dots, t_n\}$. At each layer $l \in \{1, 2, \dots, L\}$, the model produces a sequence of hidden state vectors. We denote the set of hidden representations for layer l as $H^l = \{h_{[CLS]}^l, h_1^l, \dots, h_n^l\}$, where each $h_j^l \in \mathbb{R}^{d_{model}}$ is the hidden state vector for the j -th token at layer l , and d_{model} is the hidden dimension size of the LM.

The complete set of representations extracted across all layers is $\mathcal{H} = \{H^1, H^2, \dots, H^L\}$. This comprehensive collection captures the evolution of token representations as they pass through the network, from lower layers typically capturing more surface-level features to higher layers encoding more abstract semantic and contextual information.

Step 2: Focusing on the [CLS] Token Representation

For tasks involving sentence-level understanding (e.g., sentence classification, sentiment analysis), the representation of the special [CLS] token is often used as an aggregated representation of the entire input sequence. Following this common practice, and for the reasons explained above, we focus specifically on the [CLS] token’s hidden state at each layer, $h_{[CLS]}^l$. We hypothesize that the representational characteristics of $h_{[CLS]}^l$ provide a meaningful proxy for the overall semantic information captured by layer l concerning the input sentence. Therefore, the sequence of vectors $\{h_{[CLS]}^1, h_{[CLS]}^2, \dots, h_{[CLS]}^L\}$ forms the

basis for our layer selection process.

Step 3: Quantifying Layer Representational Similarity

To determine which layers to select, we need a quantitative measure that reflects how “typical” or “unique” a layer’s representation is relative to the representations produced by other layers. For the reasons explained in Section 3.1, we propose using the average cosine similarity between a layer’s [CLS] representation and the [CLS] representations of all other layers.

The cosine similarity between two vectors u and v is defined as $\cos_sim(u, v) = \frac{u \cdot v}{\|u\| \|v\|}$, measuring the cosine of the angle between them. A value close to 1 indicates high similarity (small angle), a value close to -1 indicates high dissimilarity (large angle, near opposite direction), and a value close to 0 indicates orthogonality (uncorrelated directions).

For each layer $l \in \{1, 2, \dots, L\}$, we compute its selection score, $\text{Score}(l)$, as follows:

$$\text{Score}(l) = \frac{1}{L-1} \sum_{\substack{i=1 \\ i \neq l}}^L \cos_sim(h_{[\text{CLS}]}^l, h_{[\text{CLS}]}^i) \quad (3.1)$$

This score represents the average alignment of layer l ’s sentence representation ($h_{[\text{CLS}]}^l$) with the sentence representations from all other layers. A high score suggests that layer l ’s representation is highly congruent with the representations across the network, potentially indicating it captures more general, consensus semantic information derived from the pre-training objective. Conversely, a low score suggests that layer l ’s representation is more distinct or specialized, deviating from the average representational trend across the model’s depth.

Step 4: Layer Selection Strategies

After computing $\text{Score}(l)$ for all layers $l = 1, \dots, L$, we rank the layers based on these scores. Our goal is to select a subset of k layers (where $k < L$) for subsequent fine-tuning on a specific downstream task. The choice of which k layers to select can significantly impact the fine-tuning process and final task performance. We investigate three distinct strategies:

- **High Scoring k layers**

This strategy prioritizes layers whose [CLS] representations exhibit the highest average cosine similarity with other layers. The underlying assumption is that these high-scoring layers encapsulate the most central, generic, or foundational semantic information learned during pre-training, reflecting a strong consensus across the model’s hierarchy.

By selecting and fine-tuning these layers, we aim to adapt the core representational structures of the pre-trained model to the domain-specific attributes of the downstream task. This approach essentially modifies the most “general purpose” components of the LM. Fine-tuning these layers could be beneficial for transferring broad knowledge effectively, providing a robust starting point for adaptation. It might be particularly useful when the downstream task aligns well with the general language understanding capabilities learned during pre-training.

- **Low Scoring k layers**

This strategy takes the opposite approach, selecting the k layers with the lowest average cosine similarity scores. These layers produce [CLS] representations that

are, on average, the most distinct or dissimilar from those of other layers. This distinctiveness might indicate that these layers capture unique aspects of the input, potentially encoding more specialized features or representations that deviate significantly from the network’s overall representational trend. These might be layers where task-specific information could be more readily developed or refined.

Fine-tuning these low-scoring layers focuses the adaptation process on the more idiosyncratic parts of the model. The hypothesis is that these layers might be implicitly learning features more critical or specific to certain types of processing, which could be highly relevant to the downstream task. By concentrating fine-tuning efforts here, we might enhance the model’s ability to capture task-specific complexities, potentially leading to better performance if these layers are indeed crucial discriminators for the task.

- **Block-wise k layers**

Pre-trained LMs exhibit a hierarchical structure where different layers often capture different levels of abstraction (e.g., syntactic features in lower layers, complex semantics in higher layers). Selecting layers clustered only at the top or bottom might neglect valuable information from other parts of the network. The block-wise strategy aims to ensure a more balanced selection across the entire depth of the model. We partition the L layers into k contiguous blocks (e.g., layers 1 to L/k form the first block, $L/k + 1$ to $2L/k$ the second, and so on, with adjustments for non-divisibility). Within each block, we then select one representative layer based on its score – either the layer with the highest score within that block or the layer with the lowest score within that block.

This method guarantees that the selected layers are distributed throughout the model’s architecture, spanning shallow, intermediate, and deep layers. This ensures that the fine-tuning process influences representations at various levels of abstraction. By avoiding the selection of adjacent layers (which might redundantly adjust similar features) and preventing concentration in one region of the network depth, this strategy promotes a holistic adaptation. It allows for the refinement of both low-level and high-level features, potentially leading to a more robust and comprehensive adjustment of the model to the downstream task requirements.

The output of this selection stage is a set of k layer indices, $\mathcal{L}_{selected} \subset \{1, 2, \dots, L\}$, which are designated for parameter updates during the fine-tuning phase on the target downstream task. This targeted approach contrasts with full fine-tuning, where all L layers are updated.

3.2.2 Partial Finetuning Stage

Following the selection stage, the partial fine-tuning stage commences the actual adaptation process for the downstream task. Before training begins, the parameters of all layers not included in the selected set $\mathcal{L}_{selected}$ (i.e., the $L - k$ layers) are explicitly frozen. This means their weights and biases will not receive gradient updates during backpropagation, and their optimizer states do not need to be stored, contributing significantly to memory savings. These frozen layers effectively act as fixed, pre-trained feature extractors.

During training, for each input instance, a forward pass is executed through the entire L -layer model. Although $L - k$ layers are frozen, their computations are still necessary to produce the inputs for subsequent layers and ultimately the final output features. After the forward pass, we again extract the [CLS] token representation from each of the L layers, including both the selected (trainable) layers $l \in \mathcal{L}_{selected}$ and the unselected

Algorithm 1 k -Layer Selective Fine-tuning

- 1: **Input:** Model $M(\Theta)$ with L layers (parameters Θ_ℓ) and head (Θ_{head}); $k \in \{1, \dots, L\}$; method $f \in \{\text{'high'}, \text{'low'}\}$; Dataset $\mathcal{D}$.
- 2: **Output:** Fine-tuned model parameters Θ^* .
- 3: **Initialize:** Average score vector $\bar{\mathbf{S}} \in \mathbb{R}^L$ to $\mathbf{0}$.

▷ Selection Stage

- 4: **for all** $\mathbf{x} \in \mathcal{D}_{sample} \subseteq \mathcal{D}$ **do**
- 5: Extract $\mathbf{H}^{[CLS]}(\mathbf{x}) = \{\mathbf{h}_\ell^{[CLS]}(\mathbf{x})\}_{\ell=1}^L \leftarrow M(\mathbf{x})$.
- 6: Compute scores $\mathbf{S}(\mathbf{x}) \in \mathbb{R}^L$: $S_\ell(\mathbf{x}) = \frac{1}{L-1} \sum_{j \neq \ell} \cos_sim(\mathbf{h}_\ell^{[CLS]}(\mathbf{x}), \mathbf{h}_j^{[CLS]}(\mathbf{x}))$.
- 7: $\bar{\mathbf{S}} \leftarrow \bar{\mathbf{S}} + \mathbf{S}(\mathbf{x})$.
- 8: **end for**
- 9: $\bar{\mathbf{S}} \leftarrow \bar{\mathbf{S}} / |\mathcal{D}_{sample}|$. ▷ Final average scores
- 10: **if** $f = \text{'high'}$ **then**
- 11: $\mathcal{L}_{select} \leftarrow \arg \max_{\ell \in \{1..L\}} k(\bar{S}_\ell)$. ▷ Indices of k highest scores
- 12: **else** [$f = \text{'low'}$]
- 13: $\mathcal{L}_{select} \leftarrow \arg \min_{\ell \in \{1..L\}} k(\bar{S}_\ell)$. ▷ Indices of k lowest scores
- 14: **end if**

▷ Training Stage

- 15: Define trainable parameters: $\Theta_{train} \leftarrow (\cup_{\ell \in \mathcal{L}_{select}} \Theta_\ell) \cup \Theta_{head}$.
 - 16: Freeze parameters $\Theta_{frozen} \leftarrow \Theta \setminus \Theta_{train}$.
 - 17: Optimize: $\Theta^* \leftarrow \arg \min_{\Theta_{train}} \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}_{train}} [\mathcal{L}(M(\mathbf{x}; \Theta_{train}, \Theta_{frozen}), y)]$, where $\mathcal{L}$ is the task loss.
-

(frozen) layers $l \notin \mathcal{L}_{selected}$. This ensures that the final classifier has access to the full spectrum of representational information generated across the model’s depth, leveraging both the static knowledge from frozen layers and the dynamically adapted representations from the tuned layers.

These L individual [CLS] vectors, $\{h_{[CLS]}^1, h_{[CLS]}^2, \dots, h_{[CLS]}^L\}$, are concatenated in their layer order to form a single, high-dimensional representation vector, H_{repr} :

$$H_{repr} = \text{concat}(h_{[CLS]}^1 || h_{[CLS]}^2 || \dots || h_{[CLS]}^L) \quad (3.2)$$

The resulting vector H_{repr} has a dimensionality of $L \times d$, where d is the hidden dimension size of the transformer model.

This concatenated representation H_{repr} is then fed into a task-specific classifier head. In our experiments, this is a simple linear layer (with weight matrix W and bias vector b) followed by a softmax activation function (for classification tasks) or no activation (for regression tasks like STS-B). The classifier’s parameters (W and b) are randomly initialized before training.

The crucial aspect of this stage is the backward pass and parameter update. Gradients are computed based on the task-specific loss function (e.g., cross-entropy for classification) calculated from the classifier’s output and the ground-truth labels. However, these gradients are only used to update the parameters of the k selected layers (those indexed in set $\mathcal{L}_{selected}$) and the parameters of the classifier head (W and b). All other $L - k$ layers remain unchanged. Standard optimization algorithms like AdamW are used for these updates. This selective updating mechanism drastically reduces the number of parameters being trained (typically by around 75% for $k = 3$ in a 12-layer model) and the computational cost of the backward pass compared to full fine-tuning, directly leading to

the observed training speedups and memory efficiency. The entire process, from input processing through selection and partial fine-tuning, is summarized in Algorithm 1.

4 Baselines

4.1 Standard Baselines

4.1.1 Full Fine-tuning

As the standard baseline, we adopt a full-tuning strategy that leverages the entire architecture of the model, ensuring all parameters are adapted to the specific task at hand. Consider an input sentence S , tokenized and prepended with the special [CLS] token. This sentence is processed by the pre-trained LM, f_{LM} , which comprises L layers (e.g., 12 or 24 layers in typical Transformer-based models), each producing a contextualized representation for every token in the sequence.

For each layer l (where $l = 1, 2, \dots, L$), we extract the hidden state corresponding to the [CLS] token, $h_{[\text{CLS}]}^l$, a vector (typically of dimension 768 or 1024, depending on the model size) that encapsulates the sentence’s semantic content as interpreted by that layer. These layerwise [CLS] representations vary in nature: lower layers (e.g., $l = 1, 2$) tend to capture syntactic and local contextual features, such as word order or phrase structure, while higher layers (e.g., $l = L - 1, L$) encode more abstract, task-relevant semantic information, reflecting the hierarchical feature extraction of the Transformer architecture. To construct a rich, multi-granular sentence representation, we concatenate these [CLS] token representations from all L layers into a single vector:

$$H_{\text{repr}} = \text{concat}(h_{[\text{CLS}]}^1, h_{[\text{CLS}]}^2, \dots, h_{[\text{CLS}]}^L), \quad (4.1)$$

resulting in a high-dimensional vector of size $L \times d$ (where d is the hidden dimension, e.g., 768), which aggregates insights across the model’s depth. This concatenated representation H_{repr} is then passed to a linear classifier, a simple feedforward layer that performs a linear transformation followed by a softmax activation to predict the class probabilities:

$$\hat{y} = \text{softmax}(WH_{\text{repr}} + b), \quad (4.2)$$

where W is a weight matrix of shape $|C| \times (L \times d)$ (with $|C|$ being the number of classes) and b is a bias vector of length $|C|$. During training, the full-tuning process updates all parameters of the LM, including the embeddings, self-attention weights, and feedforward networks in all L layers, alongside the classifier parameters W and b , optimizing the cross-entropy loss between $\hat{y}$ and the true labels using gradient-based methods like Adam.

This comprehensive adaptation distinguishes full-tuning from partial fine-tuning approaches, as it allows the entire model to adjust its weights to the granularities of the downstream task, potentially enhancing performance on complex datasets by aligning low-level features and high-level abstractions alike. However, this flexibility comes with trade-offs: full-tuning demands significant computational resources (e.g., GPU memory scaling with L and sequence length), increases training time, and risks overfitting, especially on smaller datasets where the model’s vast parameter space (often hundreds of

millions of parameters) may memorize training examples rather than generalize. Thus, this full-tuning baseline serves as a powerful yet resource-intensive method, harnessing the pre-trained LM’s full capacity to achieve state-of-the-art results on classification tasks, assuming sufficient data and computational power are available.

4.1.2 LoRA

Low-Rank Adaptation (LoRA) [19] is a parameter-efficient method designed to fine-tune large pre-trained models by introducing low-rank updates to their weight matrices, thereby reducing the computational and memory demands associated with traditional fine-tuning. In standard fine-tuning, all parameters of a pre-trained model are updated, which, as mentioned, can be resource-intensive, especially for models with millions or billions of parameters. LoRA addresses this challenge by adapting the model with a small number of trainable parameters while keeping the original weights fixed.

Mathematically, consider a pre-trained weight matrix $W \in \mathbb{R}^{d \times k}$ within a neural network layer, where d is the output dimension and k is the input dimension. Instead of updating W directly, LoRA introduces an adaptation matrix ΔW , such that the effective weight matrix becomes $W' = W + \Delta W$. The key innovation of LoRA is to parameterize ΔW as a low-rank matrix, expressed as the product of two smaller matrices: $\Delta W = BA$, where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$. Here, r is a hyperparameter representing the rank of the adaptation, chosen to be much smaller than $\min(d, k)$, ensuring that ΔW has low rank.

During fine-tuning, the pre-trained weights W are frozen, and only the matrices A and B are optimized using the task-specific data. For an input vector $x \in \mathbb{R}^k$, the forward pass through the adapted layer is computed as:

$$h = W'x = (W + \Delta W)x = Wx + B(Ax) \quad (4.3)$$

This formulation leverages the linearity of matrix operations. To enhance computational efficiency, the output is typically calculated in two steps: first, Ax is computed, resulting in a vector in $\mathbb{R}^r$, and then $B(Ax)$ yields the final contribution in $\mathbb{R}^d$, which is added to Wx . This avoids explicitly forming the full $d \times k$ matrix ΔW , reducing both memory usage and computational cost.

The number of trainable parameters in LoRA is determined by the dimensions of A and B . Since A has $r \times k$ elements and B has $d \times r$ elements, the total number of parameters is:

$$r \times k + d \times r = r(d + k) \quad (4.4)$$

This is significantly smaller than the $d \times k$ parameters required to update the full weight matrix W , especially when r is small. For instance, if $d = k = 1000$ and $r = 10$, LoRA requires only $10 \times (1000 + 1000) = 20,000$ parameters compared to $1000 \times 1000 = 1,000,000$ for full fine-tuning, a reduction by a factor of 50.

In practice, the rank r is a tunable parameter that balances the trade-off between model capacity and efficiency. Small values of r (e.g., $r = 1$ or $r = 2$) have been shown to achieve performance comparable to full fine-tuning in many tasks, demonstrating that task-specific adaptations often lie within a low-dimensional subspace. Additionally, LoRA can be applied to multiple weight matrices within a model, such as those in the self-attention or feed-forward layers of a transformer, with each matrix having its own pair of A and B matrices tailored to its dimensions.

To ensure that the model starts from the pre-trained state, the matrix B is typically initialized to zero, making $\Delta W = BA = 0$ at the outset, while A may be initialized

with random values (e.g., from a Gaussian distribution). During training, gradients are computed only with respect to A and B , leaving W unchanged. This approach not only reduces the number of parameters to optimize but also enables efficient storage and computation, as the low-rank updates can either be kept separate or merged into W for inference, depending on the implementation.

By leveraging low-rank updates, LoRA provides a powerful and flexible framework for adapting large pre-trained models to new tasks, offering significant reductions in memory footprint and training time while maintaining competitive performance. This makes it particularly valuable in resource-constrained settings or when fine-tuning a single pre-trained model for multiple tasks without requiring full model copies for each, hence making it an effective baseline for our study.

4.2 Partial Finetuning Baselines

4.2.1 Random k Layers

In this baseline, we implement a straightforward approach by randomly selecting k layers from a pre-trained language model (LM) that comprises L total layers, where $k < L$, and fine-tune exclusively these chosen layers for a specific downstream classification task, while keeping the parameters of the remaining $L-k$ layers frozen throughout the training process. The selection process begins by defining the set of all layer indices as $\{1, 2, \dots, L\}$, from which we uniformly sample a subset $S \subseteq \{1, 2, \dots, L\}$ of size $|S| = k$; this uniform sampling ensures that each layer has an equal probability of being selected, calculated as k/L , and this subset S is determined once prior to training and remains fixed for the entirety of the fine-tuning procedure.

For a given input sentence S , tokenized into a sequence $\mathcal{T} = \{t_{[\text{CLS}]}, t_1, t_2, \dots, t_n\}$, where $t_{[\text{CLS}]}$ is the special classification token, we process it through the LM to generate layerwise representations H^l for each layer $l = 1$ to L . Here, $H^l = [h_{[\text{CLS}]}^l, h_1^l, \dots, h_n^l]$ represents the hidden state vectors for all tokens at layer l , with $h_{[\text{CLS}]}^l$ specifically denoting the representation of the [CLS] token at that layer.

From each of the L layers, encompassing both the k selected layers and the $L-k$ unselected ones, we extract the [CLS] token representation $h_{[\text{CLS}]}^l$, resulting in a set of L vectors. These vectors are then concatenated in order of layer index to form a single, high-dimensional representation $H_{\text{repr}} = \text{concat}(h_{[\text{CLS}]}^1 \parallel h_{[\text{CLS}]}^2 \parallel \dots \parallel h_{[\text{CLS}]}^L)$, where the concatenation operation $\parallel$ stacks the vectors into a unified feature vector whose dimensionality is the product of L and the hidden size of the LM (e.g., 768 for BERT-base). This concatenated representation H_{repr} is subsequently passed into a linear classifier defined by the equation $y = W \cdot H_{\text{repr}} + b$, where W is a weight matrix of shape $[C, L \cdot d]$ (with C being the number of classes in the downstream task and d the hidden dimension) and b is a bias vector of size C , both of which are trainable parameters initialized randomly.

During the fine-tuning phase, the optimization process updates only the parameters of the k selected layers, denoted as H^j for $j \in S$, along with the classifier parameters W and b , employing a gradient-based method such as Adam with a task-specific loss function (e.g., cross-entropy loss), while the parameters of the remaining layers H^j for $j \notin S$ are held constant at their pre-trained values, thereby reducing the computational cost compared to full model fine-tuning.

This random selection strategy, devoid of any consideration for layer-specific properties, such as their depth, representational capacity, or relevance to the downstream task, contrasts sharply with our proposed scoring-based layer selection methods, which leverage

criteria like layer importance or task alignment to guide the selection process. By fixing the randomly chosen layers before training commences and maintaining their selection throughout, this method introduces no additional complexity beyond the random sampling step, making it a computationally lightweight yet naive baseline. Consequently, the “random k layers” approach serves as a critical benchmark, enabling us to quantitatively assess whether our more informed and sophisticated layer selection strategies yield meaningful performance improvements over this simple, uninformed tactic in terms of accuracy, efficiency, or task-specific metrics on the downstream classification task.

4.2.2 Bottom k Layers

Here, we adopt a partial fine-tuning strategy that involves selectively updating only the initial k layers of a pre-trained language model, f_{LM} , while keeping the parameters of the remaining layers frozen to preserve the model’s pre-existing knowledge.

Similarly as before, when an input sentence S is provided, it is first tokenized by the model into a sequence $\mathcal{T} = \{t_{[\text{CLS}]}, t_1, t_2, \dots, t_n\}$, where $t_{[\text{CLS}]}$ is the special classification token, and n represents the total number of tokens in the sentence, determined by the tokenizer’s vocabulary and rules. The pre-trained model consists of L layers in total, each generating intermediate representations for every token in the sequence as data flows through the network.

In this approach, the “bottom k layers” specifically refer to the first k layers—namely, layers indexed from 1 to k in the forward pass—chosen for fine-tuning due to their proximity to the input and their tendency to capture more general, syntactic, and local features such as word dependencies or phrase structures, as opposed to the deeper semantic abstractions typically encoded in later layers. During the fine-tuning process for a downstream classification task, only the parameters within these initial k layers are adjusted, meaning that the weights and biases of layers $k + 1$ through L remain static, unaffected by gradient updates, which reduces computational overhead and mitigates the risk of overfitting, particularly when the target dataset is small.

For the classification task itself, we leverage the full depth of the model by extracting the [CLS] token representations from all L layers, both the fine-tuned and frozen ones, denoted mathematically as $h_{[\text{CLS}]}^1, h_{[\text{CLS}]}^2, \dots, h_{[\text{CLS}]}^L$, where each $h_{[\text{CLS}]}^l$ is a vector (of dimension d) that aggregates sentence-level information at layer l through the self-attention mechanisms inherent to transformers. These layer-specific representations are then concatenated into a single, comprehensive vector $H_{\text{repr}} = [h_{[\text{CLS}]}^1; h_{[\text{CLS}]}^2; \dots; h_{[\text{CLS}]}^L]$, resulting in a high-dimensional feature vector of size $L \times d$ that encapsulates the hierarchical processing of the input across the entire model.

This concatenated representation H_{repr} is subsequently fed into a linear classifier, a fully connected layer designed to map the $L \times d$ -dimensional input to a C -dimensional output, where C is the number of classes in the downstream task (e.g., positive/negative for sentiment analysis), followed by a softmax activation to produce class probabilities. During the training phase, gradients are computed via backpropagation to optimize a task-specific loss function, updating both the parameters of the linear classifier and the weights of the selected first k layers, while the frozen layers $k + 1$ to L serve as a fixed feature extractor, passing their pre-trained representations forward without modification.

This partial fine-tuning strategy strikes a balance between adaptability and stability, allowing the model to tailor its lower-level feature extraction to the specific task while retaining the robust, task-agnostic semantic understanding embedded in the deeper layers, a design choice that makes it computationally efficient and particularly suitable as a baseline for comparison against more sophisticated layer-selection methods explored in

this work. By focusing fine-tuning efforts on the initial layers, this method leverages the intuition that early layers, being closer to the input, encode transferable, foundational features such as token relationships or syntactic patterns that can be refined for diverse applications without requiring the resource-intensive retraining of the entire network.

4.2.3 Top k Layers

In contrast to the bottom k layers approach, the top k layers baseline method involves selectively fine-tuning the last k layers of a pre-trained transformer-based language model, f_{LM} , while keeping the initial $L - k$ layers frozen, where L represents the total number of layers in the model. In this architecture, the layers are stacked sequentially, with input data flowing from layer 1 (closest to the input) to layer L (closest to the output); thus, the “top k layers” refer to those indexed from $L - k + 1$ to L . These upper layers are widely recognized for capturing higher-level, abstract features, such as semantic relationships, syntactic nuances, and task-specific contextual information, building upon the lower-level, more general features (e.g., lexical patterns) encoded in the earlier layers.

During the fine-tuning process for a downstream classification task, only the parameters of these top k layers, denoted as $\theta_{L-k+1}, \theta_{L-k+2}, \dots, \theta_L$ (encompassing weights and biases of the multi-head self-attention mechanisms and feed-forward networks within each layer), are updated via gradient descent, while the parameters of the bottom $L - k$ layers, $\theta_1, \theta_2, \dots, \theta_{L-k}$, remain fixed, preserving the foundational language understanding acquired during pre-training. Similarly as before, for an input sentence S , the process begins with tokenization into a sequence $\mathcal{T} = \{t_{[\text{CLS}]}, t_1, t_2, \dots, t_n\}$, where $t_{[\text{CLS}]}$ is the special classification token prepended to the sequence. This sequence is then fed through all L layers, with each layer l producing a hidden representation $h^l \in \mathbb{R}^{n \times d}$ (where d is the hidden dimension), but because the bottom $L - k$ layers are frozen, their outputs remain static, serving as a stable feature extractor, while the top k layers dynamically refine their representations based on the task.

To fully exploit the model’s depth, the hidden states of the [CLS] token from all L layers, $h_{[\text{CLS}]}^1, h_{[\text{CLS}]}^2, \dots, h_{[\text{CLS}]}^L$, where each $h_{[\text{CLS}]}^l \in \mathbb{R}^d$, are extracted after processing the entire sequence. These are then concatenated into a single vector $H_{\text{repr}} = [h_{[\text{CLS}]}^1; h_{[\text{CLS}]}^2; \dots; h_{[\text{CLS}]}^L] \in \mathbb{R}^{L \times d}$, yielding a rich representation that integrates both the general linguistic knowledge from the frozen layers and the task-specific adaptations from the fine-tuned layers. This concatenated vector H_{repr} is subsequently passed to a linear classifier, implemented as a fully connected layer with weights $W \in \mathbb{R}^{C \times (L \times d)}$ and bias $b \in \mathbb{R}^C$ (where C is the number of classes), followed by a softmax activation to produce output probabilities $P(y|S) = \text{softmax}(WH_{\text{repr}} + b)$.

During training, a task-specific loss function, such as cross-entropy, is computed based on the predicted probabilities and ground-truth labels, and gradients are backpropagated exclusively through the classifier and the top k layers, updating their parameters while leaving the bottom $L - k$ layers untouched, a process that optimizes computational efficiency by reducing the number of trainable parameters (e.g., roughly k/L of the model’s total parameters).

This method leverages the top layers’ capacity to encode semantic and contextual abstractions critical for tasks like sentiment analysis or natural language inference, while retaining the robust, pre-trained features of the lower layers, making it both effective and resource-efficient. As a baseline, this approach provides a straightforward yet powerful reference for evaluating more advanced techniques, such as adaptive layer selection based on layer importance scores, gradient-based metrics, or task-specific pruning strategies, which are explored in the next sections, to further enhance performance or efficiency.

4.2.4 LIFT

LIFT (**L**ayer-**w**ise **F**ine-**T**uning), is a parameter-efficient fine-tuning technique proposed by Zhu et al. [52], aimed at reducing the computational and memory costs of adapting large pre-trained language models to specific tasks. LIFT is designed to make fine-tuning viable on resource-constrained devices by minimizing back-propagation expenses while preserving the model’s learning capacity. Unlike traditional fine-tuning, which updates all layers simultaneously, LIFT adopts a novel approach of ”learning one layer/block at a time.” This strategy not only cuts down the computational burden of the backward pass but also optimizes memory usage, making it an effective baseline for comparison in this study.

Consider a pre-trained language model $\mathcal{F}_\Theta(\cdot)$ with N transformer blocks, represented as $\mathcal{B} = \{\mathbf{B}_1, \dots, \mathbf{B}_N\}$. In standard fine-tuning, each iteration requires a forward pass through all N blocks, followed by a backward pass that computes gradients for all parameters, a process that is both computationally and memory-intensive. For instance, when using the Adam optimizer, the updates involve calculating the gradient, g_t , and maintaining first and second moments, v_t and m_t :

$$g_t = \nabla F(\theta_t); \quad v_t = \beta_1 v_{t-1} + (1 - \beta_1)g_t; \quad m_t = \beta_2 m_{t-1} + (1 - \beta_2)g_t^2 \quad (4.5)$$

This results in a memory overhead of approximately $2 \times |\Theta|$ for the optimizer states, where $|\Theta|$ is the total number of parameters. LIFT, however, selects a single block $\mathbf{B}_\ell$ per iteration, performing a full forward pass but limiting the backward pass to blocks $\mathbf{B}_N$ down to $\mathbf{B}_\ell$. This reduces the average back-propagation depth by half, cutting the backward computation cost by approximately 50%. Moreover, only the parameters of $\mathbf{B}_\ell$, denoted $|\Theta_{\mathbf{B}_\ell}|$, require optimizer states, shrinking memory usage to $2 \times |\Theta_{\mathbf{B}_\ell}|$. Unselected blocks’ optimizer states can be offloaded to the CPU and pipelined back when needed, ensuring minimal memory overhead. The block selection follows a simple front-to-end order, and each block is optimized for a fixed number of iterations, allowing all parameters to be updated over time, unlike some parameter-efficient fine-tuning methods that permanently freeze most parameters.

Beyond optimizer-related savings, LIFT further reduces memory demands by eliminating the need to store intermediate activations for blocks preceding the selected block. In standard fine-tuning, all layer activations must be retained for gradient computation during back-propagation. For a linear layer

$$\mathbf{Y}^\ell = \mathbf{W}^\ell \mathbf{x} + \mathbf{b}^\ell, \quad (4.6)$$

the gradients are:

$$\frac{\partial L}{\partial \mathbf{x}} = \frac{\partial L}{\partial \mathbf{Y}^\ell} \mathbf{W}^\ell; \quad \frac{\partial L}{\partial \mathbf{W}^\ell} = \mathbf{x}^T \frac{\partial L}{\partial \mathbf{Y}^\ell}; \quad \frac{\partial L}{\partial \mathbf{b}^\ell} = \frac{\partial L}{\partial \mathbf{Y}^\ell}, \quad (4.7)$$

requiring $\mathbf{x}$ to be stored for all learnable layers. In LIFT, since back-propagation stops at $\mathbf{B}_\ell$, activations for blocks $\mathbf{B}_1$ to $\mathbf{B}_{\ell-1}$ are not needed and can be discarded, significantly lowering memory usage, especially with large batch sizes or sequence lengths. For blocks after $\mathbf{B}_\ell$, gradients are computed, but additional savings arise from skipping gradient calculations for feed-forward network (FFN) weights beyond the selected block, reducing backward FLOPs by up to two-thirds. This combination of reduced back-propagation depth and memory efficiency makes LIFT a compelling baseline method, offering competitive performance with increased training throughput and lower resource demands, as validated in their experiments.

4.2.5 RECAP

RECAP (**RE**source and **ComputAtion**-efficient **P**runing framework) is a pruning technique introduced by Ilhan et al. [21], designed to overcome the shortcomings of traditional pruning strategies like pre-finetune and post-finetune pruning. Pre-finetune pruning prunes the full model prior to finetuning, often discarding parameters vital to the downstream task due to the absence of task-specific guidance during pruning. Post-finetune pruning, on the other hand, finetunes the full model on GPU, prunes it, and then finetunes the pruned model again, incurring high computational costs and memory demands. RECAP introduces an innovative approach by performing pruning iteratively during finetuning, allowing it to incorporate task-specific information while reducing resource usage. By finetuning only specific chunks of the model and leveraging CPU computations to guide pruning and weight updates, RECAP minimizes the GPU memory footprint associated with model weights, optimizer states, activations, and gradients, making it a resource-efficient alternative.

The RECAP methodology starts by injecting task-specific heads into a pre-trained base model, initializing the process. Its core consists of an iterative loop involving three key stages: pruning, finetuning, and updating. During the pruning stage, RECAP employs structured pruning by grouping model weights, such as attention heads and feed-forward hidden dimensions in transformers, to ensure hardware-friendly sparsity. The importance of each weight group $\mathcal{G}_m^{(p)}$ is calculated on the CPU using a small subset $\mathcal{D}_s$ of the finetuning dataset $\mathcal{D}_f$. This importance, denoted $\mathcal{I}_m^{(p)}$, is derived from a first-order Taylor expansion approximating the loss change if the group is removed:

$$\mathcal{I}_m^{(p)} = \sum_{\theta_k \in \mathcal{G}_m^{(p)}} \left(\frac{\partial \mathcal{L}(\mathcal{D}_s)}{\partial \theta_k} \theta_k \right)^2 \quad (4.8)$$

where $\mathcal{L}(\mathcal{D}_s)$ is the loss over $\mathcal{D}_s$, and $\frac{\partial \mathcal{L}(\mathcal{D}_s)}{\partial \theta_k}$ is the gradient with respect to weight θ_k . Groups are sorted by importance, and those with the lowest values are pruned iteratively over N_p steps until the desired pruning ratio r_p is achieved, preserving weights critical to the task.

Once pruning is complete, the finetuning stage determines which weights in the pruned model to update, further optimizing resource use. RECAP computes the finetuning importance $\mathcal{I}_n^{(f)}$ for each weight group $\mathcal{G}_n^{(f)}$ using the empirical Fisher information:

$$\mathcal{I}_n^{(f)} = \sum_{\theta_k \in \mathcal{G}_n^{(f)}} \left(\frac{\partial \log p_{\theta_k}(\mathcal{D}_s)}{\partial \theta_k} \right)^2 \quad (4.9)$$

This measures the impact of updating a weight on the model’s output, with higher values indicating greater sensitivity. Only a portion $v_f\%$ of weights with the highest importance are updated, reducing the need to store optimizer states and gradients for all parameters. The pruned model and a finetuning mask, specifying which weights to update, are loaded onto the GPU, where finetuning occurs over N_f steps on the downstream task. This masked update strategy not only conserves GPU memory but also prevents premature convergence by allowing the importance of pruned weights to evolve across iterations.

In the updating stage, finetuned weight updates are transferred to the CPU, overwriting the corresponding full model weights. To maintain stability and momentum across iterations, optimizer states are updated for all weights, even those not finetuned in the current round. For SGD with momentum β , the velocity V_k is adjusted as:

$$V_k \leftarrow \begin{cases} V_k & \text{if } \theta_k \text{ was updated} \\ \beta V_k & \text{otherwise} \end{cases} \quad (4.10)$$

This ensures continuity in the optimization process. The pruning, finetuning, and updating stages repeat for K iterations or until validation performance converges, iteratively refining the model. RECAP’s ability to balance task-specific pruning with resource efficiency makes it a compelling baseline for model compression in resource-constrained settings, with potential for further enhancement through techniques like quantization.

4.2.6 SlimFit

SlimFit [1] is a memory-efficient fine-tuning method tailored for large pre-trained transformer models, such as BERT and ViT, with the primary objective of reducing the memory footprint during the fine-tuning process in resource-constrained environments, such as on-device GPU settings. This method is particularly valuable for enabling the deployment of sophisticated models on devices with limited memory, where traditional fine-tuning approaches might be infeasible due to high memory demands. SlimFit achieves its efficiency by integrating selective layer freezing, quantization, and pruning techniques, guided by an analysis of the model’s training dynamics. The methodology focuses on identifying and prioritizing the layers most critical to learning a specific downstream task, thereby minimizing unnecessary memory usage and computational overhead while preserving model accuracy.

The core of SlimFit’s methodology is its use of training dynamics to assess the importance of each layer during fine-tuning. Training dynamics are quantified by measuring the changes in a layer’s parameters across training iterations, specifically using the ℓ_1 -norm of the parameter updates. This is mathematically expressed in the following equation:

$$d_i^t = \frac{1}{M \times I} \left\| \frac{\mathbf{W}_i^t - \mathbf{W}_i^{t-1}}{\mathbf{W}_i^{t-1}} \right\|_{\ell_1}, \quad (4.11)$$

where $\mathbf{W}_i^t$ represents the parameters of the i -th layer at iteration t , $M \times I$ denotes the size of the parameter matrix, and d_i^t is the distance value for that layer. The vector $\mathbf{d}^t$, comprising all d_i^t values across the model’s n layers, is termed the distance vector. Layers exhibiting small distance values indicate minimal parameter updates, suggesting they have either converged or are less relevant to the task at hand. SlimFit employs an Inter-Layer Scheduling (ILS) algorithm to leverage these distance values, dynamically ranking layers at each iteration and freezing a percentage, determined by a hyperparameter called the freezing rate, of layers with the smallest updates. Initially, the distance vector is populated with large random values, which are replaced with actual distance values over the first few iterations, after which the ILS algorithm adaptively freezes layers, updating only those with significant contributions to the learning process.

While freezing layers reduces the memory required for dynamic activations (those that can be discarded when a layer is frozen), it does not fully address the memory demands of transformer models due to an imbalance in activation counts across layers. In models like BERT and ViT, attention layers, for instance, contain up to four times more activations than other layers, undermining the expected memory savings from freezing alone. To counter this, SlimFit applies quantization to the activations of these imbalanced layers, using 8-bit quantization to compress activations where the imbalance factor is significant, thereby balancing the memory load. Additionally, SlimFit distinguishes between dynamic and static activations, the latter of which persist regardless of freezing, as found in layers

such as GELU, MatMul, Softmax, and LayerNorm. For static activations in GELU, SlimFit uses 4-bit quantization, storing two 4-bit values in an 8-bit tensor via bit-level operations due to PyTorch’s lack of native 4-bit support, while MatMul and Softmax activations are quantized to 8 bits. These quantization strategies effectively shrink the memory footprint of static activations, enhancing overall efficiency.

For layers like LayerNorm, which contain trainable parameters and static activations, SlimFit introduces a pruning technique when the layer is frozen. The forward pass of LayerNorm is defined as:

$$\tilde{\mathbf{x}} = \frac{\mathbf{x} - \mathbb{E}(\mathbf{x})}{\sqrt{\text{Var}(\mathbf{x}) + \epsilon}}, \quad (4.12)$$

$$\mathbf{y} = \tilde{\mathbf{x}} * \gamma + \beta, \quad (4.13)$$

where $\mathbf{x}$ and $\mathbf{y}$ are the input and output, and γ and β are trainable parameters. During backpropagation, the standardized input $\tilde{\mathbf{x}}$ is required, even when the layer is frozen, as seen in the derivative computation:

$$\hat{\mathbf{x}} = H * \mathbf{g} - \sum_H \mathbf{g} - \tilde{\mathbf{x}} * \sum_H (\mathbf{g} * \tilde{\mathbf{x}}), \quad (4.14)$$

where $\mathbf{g} = \frac{\gamma * \hat{\mathbf{y}}}{H * \sqrt{\text{Var}(\mathbf{x}) + \epsilon}}$. SlimFit prunes the smaller values of $\tilde{\mathbf{x}}$, retaining only the top 10% of the largest values, significantly reducing memory usage while maintaining approximate correctness in backpropagation. This transforms LayerNorm into a semi-static layer, balancing memory savings with functional integrity.

By effectively combining the ILS algorithm with quantization and pruning, SlimFit achieves a substantial reduction in the memory footprint of both dynamic and static activations during fine-tuning. The freezing rate, adjustable based on the available memory budget, dictates the extent of memory savings, with higher rates offering greater reductions at the potential cost of accuracy, depending on the task. This adaptive and multi-faceted approach allows SlimFit to fine-tune large transformer models efficiently on memory-constrained devices, making it a practical baseline for parameter-efficient fine-tuning.

4.3 I/O Similarity-based Selection Baseline

In this section, we outline two baseline methods for fine-tuning transformer models using the I/O Similarity-based Selection approach adapted from the work of He et al. [16]. Unlike the original method, which drops blocks (transformer layers) with the lowest importance scores to reduce complexity, we propose fine-tuning specific subsets of the transformer blocks based on the scores. The first baseline, termed k Low Similarity (Important), involves fine-tuning the k blocks with the highest importance scores, targeting the most transformative components of the model. The second baseline, k High Similarity (Redundant), fine-tunes the k blocks with the lowest importance scores, focusing on the seemingly redundant modules. These methods leverage the same similarity-based metric proposed by He et al., but adapt it to explore fine-tuning rather than pruning. Below, we provide descriptions of the methodologies and justifications for using both as our baselines.

4.3.1 k Low Similarity (Important)

This method selects and fine-tunes the k transformer blocks with the highest importance scores, as determined by the I/O similarity metric from He et al. [16]. The importance score for the l -th block, denoted S_B^l , is calculated as:

$$S_B^l = 1 - \text{CosineSim}(X_B^l, Y_B^l), \quad (4.15)$$

where X_B^l and Y_B^l are the input and output hidden states of the l -th block, respectively. The cosine similarity is defined as:

$$\text{CosineSim}(X, Y) = \frac{X \cdot Y}{\|X\| \|Y\|}, \quad (4.16)$$

with $X \cdot Y$ representing the dot product of the flattened vectors X and Y , and $\|X\|$ and $\|Y\|$ being their L2 norms. A high S_B^l indicates low similarity between input and output, suggesting that the block significantly transforms its input and is thus critical to the model’s functionality. We select the k blocks with the highest S_B^l values for fine-tuning, aiming to adapt the most impactful components to the downstream task.

The rationale for this approach stems from the hypothesis that blocks with high importance scores are integral to the model’s performance. By fine-tuning these blocks, we focus computational resources on updating the parameters that contribute most to feature transformation. This selective fine-tuning reduces the number of parameters adjusted compared to full model fine-tuning, offering computational efficiency while targeting the blocks most likely to enhance task-specific performance. For instance, if a block significantly alters its input, it may encode key representations that, when adapted, improve the model’s ability to generalize to new data.

Additionally, this method aligns with the model’s pre-trained architecture by building on its existing strengths. The high importance scores reflect the blocks’ established roles in the pre-training task, and fine-tuning them leverages these capabilities for the new task. This approach may also accelerate convergence, as updates are concentrated on blocks already performing substantial work, potentially requiring fewer training iterations to achieve optimal performance. In contrast to He et al.’s pruning strategy, which retains these blocks by default, our method actively enhances them, testing the hypothesis that their adaptation is key to efficient and effective fine-tuning.

4.3.2 k High Similarity (Redundant)

In this complementary baseline, we fine-tune the k blocks with the lowest importance scores, identified as redundant due to their high input-output similarity. The importance score remains:

$$S_B^l = 1 - \text{CosineSim}(X_B^l, Y_B^l), \quad (4.17)$$

where a low S_B^l indicates that Y_B^l closely resembles X_B^l , implying minimal transformation. These blocks, with high cosine similarity and thus low importance, are traditionally considered less critical, as evidenced by He et al.’s decision to prune them. However, we propose fine-tuning these k blocks to explore their potential contributions and provide a contrasting baseline to the first method.

At first glance, fine-tuning redundant blocks may seem counterintuitive—why invest resources in components that appear to contribute little? One justification is to test whether these blocks, deemed redundant in the pre-training context, can be repurposed for the downstream task. While their outputs mimic their inputs in the original model,

fine-tuning could enable them to learn task-specific features, transforming them from near-identity mappings into more useful components. This approach examines the plasticity of these blocks, assessing whether their lack of specialization makes them adaptable to new objectives.

Moreover, this method serves as an experimental control to quantify the model’s reliance on its high-importance blocks. If fine-tuning the redundant blocks yields poor performance, it reinforces the critical role of the k Low Similarity blocks, validating the importance metric. Conversely, if performance is unexpectedly strong, it challenges the assumption that high similarity between input and output equates to irrelevance, suggesting that redundancy may be context-dependent. This comparison provides a deeper understanding of the transformer’s internal dynamics, revealing whether all blocks, important or not, can be optimized effectively.

Finally, fine-tuning the k High Similarity blocks offers practical insights into resource allocation. Like the first method, it limits the number of parameters updated, maintaining computational efficiency. However, it tests an alternative strategy: enhancing the “weakest” components rather than the strongest. This baseline thus complements the first, creating a spectrum of approaches, from focusing on the most transformative blocks to the least, that enriches the analysis of fine-tuning strategies introduced in this work.

5 Experimental Design

5.1 Datasets

We present an extensive evaluation of our method and baselines encompassing 15 distinct NLP datasets, spanning both classification and regression problems. Following previous work [13], we selected 15 English language tasks for evaluation, consisting of 8 tasks that process individual sentences (single-sentence tasks) and 7 that handle sentence pairs (sentence-pair tasks). The dataset collection is drawn from multiple reputable sources: the General Language Understanding Evaluation (GLUE) benchmark [45], the Stanford Natural Language Inference (SNLI) corpus [4], and six additional well-established sentence classification datasets, namely the Stanford Sentiment Treebank with 5 classes (SST-5), Movie Reviews (MR), Customer Reviews (CR), Multi-Perspective Question Answering (MPQA), Subjectivity (Subj), and Text REtrieval Conference (TREC) questions.

This diverse set of datasets enables a comprehensive assessment of our method’s performance across various task types, domains, and input formats. To provide a clearer understanding of the evaluation scope, we categorize the datasets into two groups, single-sentence tasks and sentence-pair tasks, and describe each dataset below, highlighting their task objectives and problem types (classification or regression).

Single-Sentence Tasks

The 8 single-sentence tasks focus on processing individual sentences and are all classification problems. They include two tasks from the GLUE benchmark and six additional datasets:

- **CoLA (Corpus of Linguistic Acceptability)**: A binary classification task sourced from the GLUE benchmark. It involves determining whether a given sentence is linguistically acceptable or grammatically incorrect, based on examples drawn from linguistic publications.
- **SST-2 (Stanford Sentiment Treebank)**: A binary sentiment classification task from the GLUE benchmark. It requires predicting whether a movie review expresses a positive or negative sentiment, using a subset of the Stanford Sentiment Treebank with binary labels.
- **SST-5 (Stanford Sentiment Treebank with 5 Classes)**: A fine-grained sentiment classification task extending the SST dataset. It classifies movie reviews into five sentiment categories: very negative, negative, neutral, positive, and very positive.
- **MR (Movie Reviews)**: A binary sentiment classification task involving movie reviews collected from the IMDb archive. The goal is to classify each review as expressing a positive or negative opinion.

- **CR (Customer Reviews)**: A binary sentiment classification task based on customer reviews of products. It requires distinguishing between positive and negative sentiments expressed in short review texts.
- **MPQA (Multi-Perspective Question Answering)**: A binary classification task focused on opinion polarity detection. It involves classifying phrases or sentences from news articles and other texts as positive or negative in sentiment.
- **Subj (Subjectivity)**: A binary classification task that distinguishes between subjective sentences (expressing opinions or emotions) and objective sentences (stating facts), using data from movie review snippets and plot summaries.
- **TREC (Text REtrieval Conference)**: A multi-class classification task involving question classification. It requires categorizing questions into one of six types (e.g., location, person, number), based on data from the TREC question-answering track.

Sentence-Pair Tasks

The 7 sentence-pair tasks involve processing pairs of sentences and include a mix of classification and regression problems. They are sourced from the GLUE benchmark and the SNLI corpus:

- **MNLI (Multi-Genre Natural Language Inference)**: A 3-class classification task from the GLUE benchmark. It requires determining whether a premise sentence entails, contradicts, or is neutral with respect to a hypothesis sentence, using data from diverse genres such as fiction, government reports, and telephone conversations.
- **SNLI (Stanford Natural Language Inference)**: A 3-class classification task similar to MNLI, sourced from the SNLI corpus. It involves natural language inference with sentence pairs derived from image captions, labeled as entailment, contradiction, or neutral.
- **QQP (Quora Question Pairs)**: A binary classification task from the GLUE benchmark. It involves detecting whether two questions posted on Quora are paraphrases of each other (i.e., semantically equivalent) or not.
- **RTE (Recognizing Textual Entailment)**: A binary classification task from the GLUE benchmark. It requires determining whether a premise sentence entails a hypothesis sentence, using data from news and Wikipedia articles.
- **MRPC (Microsoft Research Paraphrase Corpus)**: A binary classification task from the GLUE benchmark. It involves identifying whether two sentences, sourced from news articles, are paraphrases of each other.
- **STS-B (Semantic Textual Similarity Benchmark)**: A regression task from the GLUE benchmark. It requires predicting a similarity score (ranging from 1 to 5) for a pair of sentences, based on their semantic equivalence, using data from news headlines, captions, and forums.
- **QNLI (Question-answering Natural Language Inference)**: A binary classification task from the GLUE benchmark. It involves determining whether a sentence contains the answer to a given question, reformulated from the Stanford Question Answering Dataset (SQuAD) as a sentence-pair task.

Table 5.1: The datasets evaluated in this work. $|\mathcal{Y}|$: number of classes for classification tasks (with one exception: STS-B is a real-valued regression task over the interval $[0, 5]$). L : average number of words in input sentence(s) [13].

Category	Dataset	$ \mathcal{Y} $	L	#Train	#Test	Type	Labels (classification tasks)
single-sentence	SST-2	2	19	6,920	872	sentiment	positive, negative
	SST-5	5	18	8,544	2,210	sentiment	v. pos., positive, neutral, negative, v. neg.
	MR	2	20	8,662	2,000	sentiment	positive, negative
	CR	2	19	1,775	2,000	sentiment	positive, negative
	MPQA	2	3	8,606	2,000	opinion polarity	positive, negative
	Subj	2	23	8,000	2,000	subjectivity	subjective, objective
	TREC	6	10	5,452	500	question cls.	abbr., entity, description, human, loc., num.
	CoLA	2	8	8,551	1,042	acceptability	grammatical, not_grammatical
sentence-pair	MNLI	3	22/11	392,702	9,815	NLI	entailment, neutral, contradiction
	SNLI	3	14/8	549,367	9,842	NLI	entailment, neutral, contradiction
	QNLI	2	11/30	104,743	5,463	NLI	entailment, not_entailment
	RTE	2	49/10	2,490	277	NLI	entailment, not_entailment
	MRPC	2	22/21	3,668	408	paraphrase	equivalent, not_equivalent
	QQP	2	12/12	363,846	40,431	paraphrase	equivalent, not_equivalent
	STS-B	$\mathcal{R}$	11/11	5,749	1,500	sent. similarity	-

The 15 datasets collectively comprise 14 classification tasks and 1 regression task (STS-B), reflecting a balanced mix of problem types. The single-sentence tasks are exclusively classification-based, covering domains such as sentiment analysis (SST-2, SST-5, MR, CR, MPQA), linguistic acceptability (CoLA), subjectivity detection (Subj), and question classification (TREC). The sentence-pair tasks include both classification (MNLI, SNLI, QQP, RTE, MRPC, QNLI) and regression (STS-B), spanning natural language inference, paraphrase detection, textual entailment, and semantic similarity.

The selection of these datasets ensures diversity in terms of task complexity, input structure (single sentences vs. sentence pairs), and application domains, which collectively challenge the generalization capabilities of our method and baselines. By incorporating tasks from the widely-adopted GLUE benchmark, we align with standard evaluation practices in NLP, while the addition of the SNLI corpus and six supplementary single-sentence datasets broadens the scope beyond GLUE, providing a more robust evaluation framework. Notably, we exclude the WNLI (Winograd Natural Language Inference) task from GLUE due to its small size and the difficulty many models face in achieving meaningful performance on it, a common practice in NLP evaluations.

For detailed statistics on each dataset, such as the number of training and test examples, refer to Table 5.1. This comprehensive dataset collection forms the basis of our experimental setup, allowing a thorough analysis of our method’s strengths and limitations across varied linguistic aspects.

5.2 Models

We utilize several publicly available transformer-based models, accessed through the HuggingFace library (version 4.45.1) and implemented using the PyTorch framework (version 2.3.0). Specifically, we employ the BERT model in its base configuration for our main results, while additional results in our ablation study include RoBERTa, DeBERTa, and ELECTRA in their base configurations, as well as the larger Gemma-2-2B and TinyLlama-1.1B models. Each model is selected for its unique architectural features and pre-training methodologies, allowing us to evaluate their effectiveness across our classification tasks. Below, we provide detailed descriptions of each model, including their architecture and the specific techniques employed in pre-training.

BERT

For our main experiments, we employ BERT (Bidirectional Encoder Representations from Transformers) in its base configuration. Introduced by Devlin et al. [11], BERT is a pioneering model pre-trained on a large corpus of text using two key objectives: Masked Language Modeling (MLM), where a portion of input tokens is masked and the model predicts them based on surrounding context, and Next Sentence Prediction (NSP), which trains the model to determine if two sentences are consecutive in the original text.

BERT’s bidirectional nature, enabled by the MLM objective, allows it to consider both left and right context when predicting masked tokens, which presented a significant leap forward from previous unidirectional models. This bidirectionality results in richer, more refined contextual representations, which are essential for tasks requiring a deep understanding of language, such as natural language inference or named entity recognition. The base configuration of BERT consists of 12 transformer layers, each with 12 attention heads and a hidden size of 768, resulting in approximately 110 million parameters. These architectural parameters (the number of layers, attention heads, and hidden size) enhance the model’s capacity to capture intricate patterns in text, though they also increase computational demands, representing a practical compromise between performance and efficiency.

RoBERTa

Developed by Liu et al. [27], RoBERTa (A Robustly Optimized BERT Pretraining Approach) builds on BERT by refining its pre-training process. It eliminates the NSP task, which research showed had limited impact on downstream performance, and focuses exclusively on the MLM objective with dynamic masking, where the masking pattern changes with each training epoch. This dynamic approach exposes the model to diverse masked versions of the same input, potentially improving its generalization across different linguistic contexts. RoBERTa also benefits from training on significantly larger datasets, such as the combination of BookCorpus, English Wikipedia, Common Crawl News, OpenWebText, and Stories, with bigger batch sizes and extended training durations. These enhancements enable RoBERTa to outperform BERT on many benchmarks, demonstrating the value of optimized pre-training strategies. Its base configuration mirrors BERT-base, featuring 12 transformer layers, 12 attention heads, and a hidden size of 768.

DeBERTa

Introduced by He et al. [15], DeBERTa (Decoding-enhanced BERT with Disentangled Attention) enhances BERT through two key innovations. First, it employs a disentangled attention mechanism, representing each word with separate content and position vectors. This allows the model to compute attention weights based on both content-to-content and content-to-position interactions, improving its ability to capture positional relationships in text, which is an advantage for tasks where word order is critical, such as machine translation or dependency parsing. Second, it incorporates an enhanced mask decoder that integrates absolute positional information into the decoding layer during the MLM pre-training process. This addition refines the model’s predictions for masked tokens by leveraging both relative and absolute positional contexts. The base configuration of DeBERTa aligns with BERT-base, consisting of 12 transformer layers, 12 attention heads, and a hidden size of 768.

ELECTRA

Proposed by Clark et al. [10], ELECTRA (Efficiently Learning an Encoder that Classifies Token Replacements Accurately) introduces a more sample-efficient pre-training approach compared to BERT. Instead of masking tokens, ELECTRA replaces some tokens with plausible alternatives generated by a small auxiliary generator model, then trains a discriminator to identify which tokens were replaced. This method enables the model to learn from all input tokens, not just the masked ones, maximizing data utilization and potentially leading to more robust representations. The use of a smaller generator, trained jointly with the discriminator, allows ELECTRA to focus on distinguishing real tokens from generated ones, enhancing its efficiency. The base configuration of ELECTRA includes 12 transformer layers, 12 attention heads, and a hidden size of 768, consistent with BERT-base.

Gemma 2 2B

The Gemma 2 2B model [39] represents the smallest entry in the second generation of Google’s Gemma family of open-source language models, specifically designed as a lightweight yet powerful 2 billion parameter decoder-only transformer, contrasting with its larger 9B and 27B variants. Built upon the foundational Transformer architecture, it features 26 layers, a model dimension of 2304, and an expanded feedforward dimension of 18432 which utilizes the approximated GeGLU non-linearity for computations.

Several key technical modifications distinguish its architecture: it employs Grouped-Query Attention (GQA) configured with 8 query heads and 4 key/value heads and a head size of 256, chosen to enhance inference speed while preserving downstream task performance. Furthermore, it interleaves different attention mechanisms across its layers, alternating between local sliding window attention with a 4096-token window and global attention spanning the full context length of 8192 tokens. Positional information is encoded using Rotary Position Embeddings (RoPE), and training stability is enhanced through the use of RMSNorm applied both before (pre-norm) and after (post-norm) each transformer sub-layer (attention and feedforward). Additionally, logit soft-capping is applied within attention layers (cap value 50.0) and at the final output layer (cap value 30.0).

The model utilizes tied input and output embeddings and incorporates the large 256k-entry SentencePiece vocabulary inherited from the Gemini models, featuring split digits, preserved whitespace, and byte-level encodings. Crucially, unlike the 27B model trained via standard next-token prediction, the Gemma 2 2B model (along with the 9B variant) was trained using knowledge distillation over 2 trillion tokens; this involved learning from the richer output probability distributions of a larger teacher model rather than just the one-hot target vector of the next token, aiming to simulate training on significantly more data than physically processed and provide stronger gradients, ultimately boosting performance for its practical size.

TinyLlama 1.1B

TinyLlama 1.1B [51] is a compact language model with 1.1 billion parameters, designed as an open-source alternative, building upon the architecture and tokenizer of Llama 2. Architecturally, it is a decoder-only Transformer featuring 22 layers, a hidden dimension of 2048, an intermediate hidden size of 5632, and 32 attention heads within a context length of 2048, using a vocabulary size of 32,000.

Key architectural choices include Rotary Positional Embedding (ROPE) for injecting positional information, pre-normalization combined with RMSNorm for training stability and efficiency, the SwiGLU activation function (a combination of Swish and Gated Linear Units) instead of standard ReLU, and Grouped-Query Attention (GQA) with 32 query heads and 4 key-value head groups to reduce memory bandwidth overhead and speed up inference.

The model was pre-trained using an autoregressive language modeling objective on up to 2 trillion tokens (for version 1.1) derived from a blend of SlimPajama and StarCoder datasets. The v1.1 training adopted a three-stage process: basic pre-training on 1.5 trillion SlimPajama tokens, continual pre-training for 350 billion tokens incorporating domain-specific corpora (like code/math or Chinese data, depending on the model variant) alongside SlimPajama with adaptive data sampling, and a final 150 billion token cooldown phase where the batch size was increased significantly (from 1.8M to 7.2M tokens) while maintaining the learning rate schedule. To optimize training speed and efficiency, the project integrated Fully Sharded Data Parallel (FSDP, sharding parameters within nodes for v1.1), FlashAttention-2, fused layernorm, fused cross-entropy loss, fused rotary positional embeddings, and the fused SwiGLU module from xFormers, achieving significant throughput on A100 GPUs using the lit-gpt framework.

5.3 Hyperparameters

To determine the optimal settings, particularly for parameters known to significantly impact convergence and generalization, we performed a grid search over a defined range of key hyperparameters. This primarily focused on learning rates and the number of training epochs, as these often exhibit strong interdependencies and dataset-specific optima. The batch size was also varied within constraints imposed by available computational resources. Crucially, the final set of hyperparameters was selected independently for each dataset, ensuring that model performance was maximized respective to the specific characteristics of the data.

Learning Rate. The learning rate, which governs the step size during the gradient descent optimization process, was a primary focus of our tuning efforts. We explored values within the set $1e-5$, $2e-5$, $5e-5$. This range is commonly employed for fine-tuning large pre-trained language models, balancing the need for sufficient adaptation to the downstream task against the risk of catastrophic forgetting or unstable training dynamics. The learning rate that yielded the best performance on the validation set for each specific dataset was chosen for the final model training and evaluation.

Training Epochs. The number of training epochs dictates how many times the model processes the entire training dataset. We evaluated model performance after 3, 5, and 10 epochs. This range allowed us to assess the learning trajectory and identify a point that balances sufficient exposure to the training data (avoiding underfitting) with the risk of overfitting to the training set, which could harm generalization performance. Similar to the learning rate, the optimal number of epochs was determined on a per-dataset basis through validation performance.

Batch Size. The batch size determines the number of training examples utilized in one iteration to estimate the gradient. We experimented with batch sizes of 16 and 32. This choice is often constrained by GPU memory limitations. Larger batch sizes can lead to more stable gradient estimates but require more memory, while smaller batch sizes introduce more noise but can sometimes offer regularization benefits and faster iteration times. The final batch size was selected based on a combination of validation performance

and hardware constraints for each dataset scenario.

Optimizer Configuration. We employed the AdamW optimizer for all model training. AdamW is a variant of the Adam optimizer that decouples weight decay from the adaptive learning rate mechanism, which has been shown to improve regularization and generalization in transformer models. We utilized standard, widely accepted settings for its momentum parameters: β_1 was set to 0.9 and β_2 was set to 0.99. An epsilon value of $1e-8$ was used for numerical stability.

Regularization. To mitigate overfitting, weight decay regularization was applied during training. A weight decay coefficient of 0.01 was consistently used across all experiments. This value penalizes large weights in the model, encouraging simpler models that often generalize better to unseen data. The weight decay was implemented through the AdamW optimizer.

LoRA Configuration. For experiments involving the Low-Rank Adaptation (LoRA) fine-tuning method, specific parameters governing the low-rank decomposition were configured. We applied LoRA to the query and value projection matrices within the self-attention mechanisms of the transformer model. The rank of the decomposition, r , was set to 16. This parameter controls the capacity of the adaptable LoRA matrices. The scaling factor, α , which modulates the magnitude of the adaptation, was set to 32. This combination of $r=16$ and $\alpha=32$ is a common configuration found effective in prior LoRA applications.

Reproducibility. To account for the inherent stochasticity in training deep learning models (e.g., due to weight initialization and data shuffling), and to ensure the reliability of our results, all experiments were conducted three times with different random seeds. The specific seeds used were [0, 42, 100]. Performance metrics reported in this work typically represent the average across these three independent runs, providing a more robust assessment of model performance.

5.4 Evaluation Metrics

To quantitatively assess the performance of our models across the diverse set of 15 benchmark datasets employed in this study, we utilized standard evaluation metrics appropriate for the specific task associated with each dataset. The selection of these metrics aligns with common practices within the natural language processing community for evaluating performance on these specific benchmarks. The metrics used are detailed below.

Accuracy

For the majority of the classification tasks, accuracy was selected as the primary evaluation metric. This includes the sentiment analysis datasets (SST-2, SST-5, MR, CR, MPQA, Subj), topic classification (TREC), grammatical acceptability (CoLA), and natural language inference tasks (MNLI, SNLI, QNLI, RTE). Accuracy measures the proportion of correctly classified instances out of the total number of instances evaluated. It provides a straightforward measure of overall correctness, particularly suitable for balanced datasets or tasks where all classes are of equal importance. It is calculated using the counts of True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN) as follows:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (5.1)$$

Where TP represents the number of positive instances correctly classified as positive, TN represents the number of negative instances correctly classified as negative, FP represents the number of negative instances incorrectly classified as positive, and FN represents the number of positive instances incorrectly classified as negative. For multi-class problems, this generalizes to the total number of correctly classified instances divided by the total number of instances.

F1 Score

For the paraphrase identification tasks, specifically MRPC and QQP, we report the F1 score. These datasets often involve assessing similarity or equivalence between sentence pairs, where simply measuring accuracy might be misleading, especially when dealing with imbalanced class distributions, which can be common in paraphrase detection where non-paraphrase pairs might significantly outnumber paraphrase pairs. The F1 score represents the harmonic mean of Precision and Recall, offering a balanced evaluation that considers both the model’s ability to avoid false positives (Precision) and its ability to find all relevant positive instances (Recall). The F1 score is calculated as follows:

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (5.2)$$

where *Precision* and *Recall* are defined as:

$$Precision = \frac{TP}{TP + FP} \quad (5.3)$$

$$Recall = \frac{TP}{TP + FN} \quad (5.4)$$

Here, TP , FP , and FN represent True Positives (correctly identified paraphrase pairs), False Positives (pairs incorrectly identified as paraphrases), and False Negatives (paraphrase pairs missed by the model), respectively.

Pearson Correlation Coefficient

For the semantic textual similarity regression task, STS-B, the performance is evaluated using the Pearson correlation coefficient, r . This dataset requires models to predict a continuous similarity score (typically between 0 and 5) reflecting the degree of semantic similarity between two sentences, rather than making a categorical classification. The Pearson correlation coefficient measures the linear correlation between the predicted similarity scores generated by the model and the human-annotated gold standard scores. A value closer to 1 indicates a strong positive linear relationship, meaning the model’s predictions align well with human judgments. The Pearson correlation coefficient is calculated using the formula:

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \cdot \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}} \quad (5.5)$$

In this equation, n is the total number of sentence pairs in the evaluation set, x_i represents the predicted similarity score for the i -th pair, y_i represents the corresponding gold standard human score, $\bar{x}$ is the mean of all predicted scores, and $\bar{y}$ is the mean of all gold standard scores. This metric effectively captures how well the model’s predicted similarity rankings match the ground truth.

6 Results and Analysis

6.1 Results

Table 6.1: Results for Single Sentence Tasks. We compare our method with full fine-tuning, LoRA fine-tuning, partial fine-tuning, LIFT, RECAP, SlimFit, and input-output similarity baselines. We use $k=3$ for all layer selection baselines. All the results are the mean of 3 runs with different seeds [0, 42, 100]. **Bold** indicates the best-performing model and underline indicates the second best.

<i>Single Sentence Tasks</i>									
	SST-2 (acc)	SST-5 (acc)	MR (acc)	CR (acc)	MPQA (acc)	Subj (acc)	TREC (acc)	CoLA (acc)	Avg.
Full Fine-tuning	91.7	53.7	87.8	88.0	88.4	96.8	96.7	82.4	85.7
LoRA Fine-tuning	88.6	50.9	85.8	85.2	86.1	92.6	92.8	80.3	82.8
Partial Finetuning Baselines [$k = 3$]									
Random k layers	88.4	50.2	85.4	84.7	85.5	93.3	95.2	79.6	82.8
Bottom k layers	88.2	50.8	85.1	84.8	85.4	93.2	94.1	79.2	82.6
Top k layers	89.0	51.2	86.6	85.5	86.8	94.9	94.2	80.4	83.6
LIFT [52]	90.4	50.5	86.8	86.1	87.0	95.9	95.8	80.5	84.1
RECAP [21]	90.3	51.2	86.3	86.7	87.3	96.2	96.3	81.0	84.4
SlimFit [1]	90.1	51.8	87.5	87.2	87.8	<u>96.3</u>	96.1	79.5	84.5
I/O Similarity-based Selection [$k = 3$]									
k Low Similarity (Important) [16]	89.1	51.3	86.3	85.2	86.0	95.2	94.0	80.0	83.4
k High Similarity (Redundant) [16]	89.8	51.8	86.8	85.5	86.8	95.1	95.4	80.6	84.0
k-Layer Selection (Ours) [$k = 3$]									
High Scoring k layers	90.6	<u>52.5</u>	88.1	86.0	87.1	96.2	96.2	81.0	84.7
Low Scoring k layers	90.8	52.9	87.0	86.7	87.7	96.2	96.8	80.8	<u>84.9</u>
Block-wise highest layer	<u>90.7</u>	<u>52.5</u>	<u>88.0</u>	86.7	88.2	96.2	96.4	<u>81.1</u>	85.0
Block-wise lowest layer	90.4	52.3	87.7	<u>86.9</u>	<u>88.1</u>	96.4	<u>96.7</u>	81.2	85.0

6.1.1 Single Sentence Tasks

We compare our method with the baselines described in Chapter 4 on the 8 single-sentence tasks (SST-2, SST-5, MR, CR, MPQA, Subj, TREC, CoLA) in Table 6.1. Our proposed strategies demonstrate strong performance across all these diverse datasets.

Performance of Proposed Strategies: The results clearly indicate the effectiveness of employing inter-layer [CLS] token similarity for layer selection. Our Block-wise lowest layer and Block-wise highest layer strategies emerge as the top performers, both achieving an average accuracy of 85.0%. This demonstrates that the Block-wise approach ensures selected layers span different stages of representation abstraction (from shallow to deep), preventing over-specialization in one region and promoting a more holistic adaptation. Selecting the highest-scoring layer within a block thus serves to refine more generic, broadly applicable representations identified by high similarity scores across the model.

Conversely, selecting the lowest-scoring layer acts to target unique, seemingly task-critical representations highlighted by low similarity, allowing for focused adaptation of specialized knowledge within that block. The strong performance of Low Scoring k layers (average accuracy 84.9%) further supports the idea that layers exhibiting distinct [CLS] representations (low similarity) are crucial for task-specific adaptation, as they likely encode unique features deviating from the model’s general knowledge. The High Scoring k layers (average accuracy 84.7%) also performs well, suggesting that refining the most consensus-driven, generic representations across layers can also be an effective strategy for aligning the model with downstream tasks, helping in enhancing generalization.

Comparison with LoRA: When compared to the popular parameter-efficient fine-tuning method LoRA, our best-performing strategies (Block-wise highest/lowest) show a substantial improvement of approximately 2.2% on average across the single-sentence tasks. LoRA applies low-rank updates to attention matrices across all layers, assuming useful adaptation can occur uniformly. In contrast, our method identifies and focuses computational effort only on a subset of layers deemed most relevant based on inter-layer [CLS] similarity dynamics. This targeted approach appears more effective, suggesting that not all layers benefit equally from adaptation, and focusing updates on strategically selected layers (whether capturing generic or specific information) yields better task alignment than distributing smaller updates universally, especially for single-sentence tasks where overall sentence semantics captured by [CLS] are integral for prediction.

Comparison with Heuristic Partial Fine-tuning: Our method consistently outperforms simple heuristic baselines. Compared to selecting a random set of $k = 3$ layers, our best strategies offer a 2.2% improvement on average. Against selecting the bottom $k = 3$ layers or the top $k = 3$ layers, our approaches show advantages of 2.4% and 1.4% on average, respectively. These heuristic approaches lack a principled basis for selection; random selection is arbitrary, while top/bottom selection relies solely on layer position, ignoring the functional specialization and inter-dependencies between layers. Our method, grounded in the semantic similarity of the [CLS] token across layers, provides a more informed selection criterion. It explicitly considers how information transforms throughout the model depth, allowing it to identify layers that are either central to the model’s overall representation (high similarity) or uniquely transformative (low similarity), leading to more effective partial fine-tuning compared to position-based or random guesses.

Comparison with Dynamic/Gradient-based Methods: We also compare favorably against more sophisticated partial fine-tuning techniques. Our top strategies outperform LIFT by 0.9%, RECAP by 0.6%, and SlimFit by 0.5%, on average. These dynamic methods typically rely on computationally heavier processes involving gradient analysis or iterative updates during training to select or freeze layers. Our method achieves comparable or better results using a training-free selection stage based solely on a forward pass and [CLS] similarity calculation. This suggests that the static inter-layer relationships captured by [CLS] similarity in the pre-trained model provide a sufficiently strong signal for effective layer selection, avoiding the overhead and complexity associated with dynamic, training-dependent selection mechanisms while achieving robust performance.

Comparison with I/O Similarity Selection: Our method also shows an advantage over selection based on input-output similarity as analyzed by He et al. [16]. Fine-tuning the $k = 3$ layers with the lowest I/O similarity (deemed important) yielded 83.4% average accuracy, while tuning the $k = 3$ highest I/O similarity layers (deemed redundant) achieved 84.0%. Our best strategies outperform the better of these (High Similarity) by a clear 1.0%. I/O similarity focuses on the representational stability within a single layer, identifying layers that cause minimal change as redundant. Our method, however, focuses on the similarity of the summarizing [CLS] token between different layers. This inter-layer

perspective seems to capture more pertinent information about the model’s hierarchical structure and which layers contribute most significantly (either through consensus or unique transformation) to the final sentence representation relevant for classification tasks, making it a more effective criterion for selecting layers to fine-tune.

Comparison with Full Fine-tuning: Crucially, the performance of our best strategies is remarkably close to that of full fine-tuning, differing by only 0.7% on average. This indicates that a large portion of the pre-trained model’s parameters might already be well-suited for downstream tasks or even prone to overfitting if fully tuned. By selectively fine-tuning only 25% of the layers identified via [CLS] similarity, our method effectively adapts the model while preserving conceivably beneficial pre-trained knowledge and reducing the risk of catastrophic forgetting or overfitting on task-specific data. The fact that some variants even occasionally outperformed full fine-tuning (e.g., on MR, TREC) further suggests that freezing certain layers can act as a beneficial regularization mechanism.

Table 6.2: Results for Sentence Pair Tasks. We compare our method with full fine-tuning, LoRA fine-tuning, partial fine-tuning, LIFT, RECAP, SlimFit, and input-output similarity baselines. We use $k=3$ for all layer selection baselines. All the results are the mean of 3 runs with different seeds [0, 42, 100]. **Bold** indicates the best-performing model and underline indicates the second best.

<i>Sentence Pair Tasks</i>								
	MNLI (acc)	SNLI (acc)	QNLI (acc)	RTE (acc)	MRPC (F1)	QQP (F1)	STS-B (Pear.)	Avg.
Full Fine-tuning	84.0	90.8	90.9	62.0	81.8	90.5	86.2	83.7
LoRA Fine-tuning	80.5	88.2	84.9	56.3	72.1	88.3	83.4	79.1
Partial Finetuning Baselines [$k = 3$]								
Random k layers	80.2	87.6	86.9	59.7	70.6	87.2	80.5	79.0
Bottom k layers	80.3	87.4	85.7	58.8	71.4	87.9	81.4	79.0
Top k layers	81.6	88.9	87.8	60.9	74.1	88.8	83.0	80.7
LIFT [52]	81.5	89.3	87.6	59.2	74.2	88.4	83.7	80.6
RECAP [21]	81.3	89.1	87.5	61.3	75.0	89.2	<u>84.4</u>	81.1
SlimFit [1]	<u>82.2</u>	89.2	88.0	58.5	75.9	87.8	84.3	80.8
I/O Similarity-based Selection [$k = 3$]								
k Low Similarity (Important) [16]	80.8	88.7	88.1	60.4	72.2	88.8	82.0	80.1
k High Similarity (Redundant) [16]	81.3	88.9	88.5	60.8	73.4	88.6	81.2	80.4
k-Layer Selection (Ours) [$k = 3$]								
High Scoring k layers	82.1	90.1	90.2	61.0	75.0	89.0	82.4	<u>81.4</u>
Low Scoring k layers	82.5	90.0	89.4	61.4	72.5	89.0	83.3	81.2
Block-wise highest layer	81.8	89.7	88.7	<u>61.7</u>	<u>75.3</u>	89.8	84.5	81.6
Block-wise lowest layer	82.0	<u>90.0</u>	<u>90.1</u>	62.0	71.6	<u>89.7</u>	83.3	81.2

6.1.2 Sentence Pair Tasks

In Table 6.2, we exhibit the results for the 7 sentence-pair tasks (MNLI, SNLI, QNLI, RTE, MRPC, QQP, STS-B) trained using our method along with the baselines. We observe a similar pattern of strong performance from our proposed strategies.

Performance of Proposed Strategies: Our approach continues to demonstrate its value on tasks requiring understanding relationships between sentence pairs. The Block-wise highest layer selection strategy yields the best average performance among the baselines at 81.6%. The High Scoring k layers approach follows closely at 81.4%, while the Block-wise lowest layer and Low Scoring k layers strategies both achieve 81.2% on

average. The consistent success of Block-wise selection reinforces the value of distributing tuned layers across the network’s depth to capture varying levels of abstraction, which is likely crucial for understanding the complex relationships between sentence pairs. The slight edge of targeting higher-scoring layers (Block-wise highest, High Scoring k) in the average might suggest that refining the core, consensus representations is marginally more beneficial for these relational tasks than focusing solely on the most unique layers identified by low scores.

Comparison with LoRA: Compared to LoRA fine-tuning, our best strategy (Block-wise highest) provides an average improvement of 2.5%. This slightly larger margin compared to single-sentence tasks might indicate that the focused adaptation provided by our layer selection is even more crucial when the task involves complex inter-sentence reasoning. LoRA’s uniform, low-rank updates might be less effective at capturing the specific, interdependent layer interactions needed to compare and contrast sentence pairs compared to concentrating full updates on a few strategically chosen layers identified through their [CLS] token relationships.

Comparison with Heuristic Partial Fine-tuning: Our method again show clear advantages over predefined heuristic strategies. The best strategy surpasses Random k layers and Bottom k layers by 2.6%, and outperforms the Top k layers baseline by 0.9% on average. Sentence-pair tasks often require integrating information processed at different representational depths (e.g., lexical details from early layers, semantic relationships from later layers). Heuristic methods based on position or randomness fail to account for this functional distribution. Our [CLS] similarity approach, by considering inter-layer relationships, is better equipped to select a set of layers that collectively contribute the necessary range of information processing for effective sentence-pair analysis.

Comparison with Dynamic/Gradient-based Methods: Our best approach also maintains an edge over dynamic selection methods: it outperforms LIFT by 1.0%, SlimFit by 0.8%, and RECAP by 0.5% on average for these sentence-pair tasks. The training-free nature of our selection, based on the inherent structure revealed by [CLS] similarity, continues to prove effective. For sentence-pair tasks, the static relationships between layer representations in the pre-trained model seem to provide a robust signal for which layers are most critical to adapt, rivaling or exceeding the performance of methods that rely on observing dynamics during the fine-tuning process itself.

Comparison with I/O Similarity Selection: Similar to single-sentence tasks, our method outperforms I/O similarity-based selection. The best I/O similarity baseline (High Similarity) achieved 80.4% on average, which is 1.2% lower than our best performing strategy. This further reinforces that assessing the relationship between layer representations (specifically the summary [CLS] token) provides a more useful signal for fine-tuning than assessing the stability within each layer independently, especially for tasks demanding integration of information across layers like sentence-pair analysis.

Comparison with Full Fine-tuning: While still competitive, the gap between our best partial fine-tuning strategy and full fine-tuning is 2.1% on average for sentence-pair tasks. This is slightly larger than the 0.7% gap observed for single-sentence tasks, indicating that the complex reasoning required for relating sentence pairs likely benefits more from the fine-grained adjustments possible across a larger number of layers in full fine-tuning. However, achieving performance within 2% of the fully fine-tuned model while only updating 25% of the parameters still represents a highly favorable trade-off, demonstrating the considerable efficiency and effectiveness of our selective fine-tuning approach even on these more challenging tasks.

Table 6.3: Comparison on number of trainable parameters and average speedup. We take the average of the speedup factor of all four of our layer selection methods. We set $k=3$.

Method	Avg Speedup $\uparrow$	#Params $\downarrow$
Full Fine-tuning	-	116.6 M
LoRA Fine-tuning	1.63	1.84 M
LIFT	0.53	29.7 M
RECAP	1.39	32.1 M
SlimFit	1.21	40.7 M
I/O Similarity-based	1.40	31.1 M
Random Selection	1.47	<u>28.3 M</u>
Our Method	<u>1.54</u>	<u>28.3 M</u>

6.1.3 Training Time and Parameters

Table 6.3 provides a comparative overview of computational efficiency, focusing on average training speedup and the number of trainable parameters for $k = 3$ selected layers. While LoRA fine-tuning demonstrates the highest speedup ($1.63\times$) due to its minimal parameter overhead (1.84 M), our proposed method achieves a highly competitive average speedup of $1.54\times$. This slight difference in speedup is counterbalanced by the notable performance improvements our method delivers across tasks (as evidenced in Tables 6.1 and 6.2, often exceeding LoRA by 2-3% absolute accuracy). This highlights a crucial trade-off: our approach offers a more favorable balance between computational efficiency and task performance, making it particularly valuable when final accuracy is the priority.

The parameter efficiency of our method is substantial, utilizing only 28.3 M trainable parameters (for $k = 3$), which constitutes approximately a 75% reduction compared to the 116.6 M parameters involved in full fine-tuning. This significant reduction directly translates to lower memory requirements during training, particularly for storing optimizer states, which can be a major bottleneck in resource-constrained environments. Furthermore, freezing the majority ($L - k = 9$) of the transformer layers means that gradient computations and parameter updates during backpropagation are restricted to only the selected k layers and the final classification head, inherently reducing the computational workload per training step and contributing to the observed speedup.

Compared to other partial fine-tuning baselines beyond LoRA, our method maintains a strong efficiency profile. It requires fewer parameters than SlimFit (40.7 M) and achieves a better speedup than both LIFT ($0.53\times$) and SlimFit ($1.21\times$). While its speedup is comparable to RECAP ($1.39\times$) and Random Selection ($1.47\times$), our approach consistently demonstrates superior performance, suggesting its layer selection strategy is more effective. A key advantage contributing to its practical efficiency is the training-free nature of the layer selection stage. Unlike methods such as SlimFit or RECAP, which may incur computational overhead during training for dynamic analysis (e.g., gradient contributions, Fisher information), our method determines the layers to fine-tune via a single, initial forward pass, simplifying the overall fine-tuning pipeline. It’s important to note that these efficiency metrics are based on $k=3$; the number of trainable parameters and the training speedup would naturally scale as k varies, as explored in our ablation studies (Section 6.3).

6.1.4 Layer Selection Insights

To better understand the layer selection mechanism, Figure 3 presents the average inter-layer cosine similarity scores based on [CLS] token representations, aggregated across all

evaluated datasets for the BERT-base model. The graph reveals a relatively scattered distribution, indicating that, on average, no specific set of adjacent layers consistently exhibits exceptionally high or low similarity scores compared to the rest. There isn’t a strong monotonic trend, suggesting that layers contributing significantly to the overall representation (higher average similarity) or undergoing unique transformations (lower average similarity) are distributed throughout the model’s depth. This average pattern supports the hypothesis that important layers for downstream tasks are not necessarily confined to the top or bottom sections of the model, reinforcing the rationale for a selection method that considers the entire network.

However, this averaged view fails to capture the dynamics at the sample level. As mentioned, the scores used for selection are calculated per-sample (or per-batch) during the initial forward pass. At this granularity, the similarity scores exhibit greater variance, meaning the relative importance or similarity profile of layers can shift depending on the specific input sentence. This input sensitivity, captured by the [CLS] token’s evolving representation and its similarity to other layers’ [CLS] representations, allows the method to identify layers most pertinent to the input’s characteristics, even though the selection mechanism itself doesn’t involve training updates.

These insights inform our different selection strategies. Selecting ‘High Scoring’ layers might favor layers whose representations are, on average or for specific inputs, more aligned with the global context captured across other layers, thereby refining more foundational aspects of the representation. Conversely, selecting ‘Low Scoring’ layers targets layers where the [CLS] representation diverges most significantly from others, indicating points of crucial task-specific abstraction or transformation that benefit most from fine-tuning. The ‘Block-wise’ strategy directly leverages the scattered nature observed in Figure 3, ensuring that the fine-tuning process incorporates layers from different representational stages (e.g., early, mid, late layers), preventing over-specialization in one part of the network and promoting a more balanced adaptation. This content-aware selection based on inter-layer relationships, sensitive to input variations yet computationally inexpensive, distinguishes our approach from fixed positional heuristics (like top- k or bottom- k freezing) and contributes to its robust performance across diverse tasks and domains. The cosine similarity scores effectively act as a proxy for representational stability versus change across layers, enabling targeted fine-tuning.

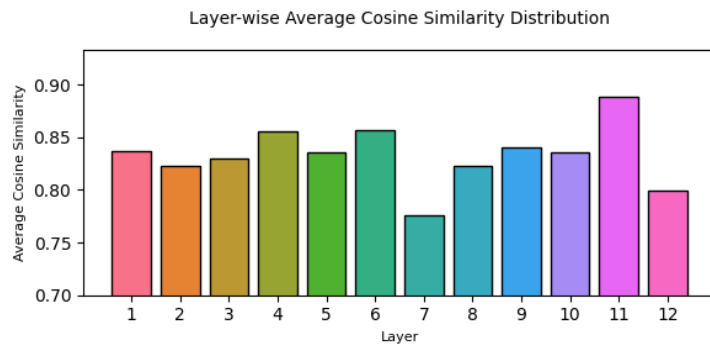


Figure 3: Distribution of layer-wise cosine similarity scores, averaged across all evaluated datasets. During the selection stage, these scores are calculated on a per-sample basis for each dataset so the actual distribution for each sample exhibits greater variance and more pronounced fluctuations than this averaged view.

6.2 Cross-Domain Evaluation

To rigorously assess the domain generalization capability and out-of-distribution (OOD) performance of our partial fine-tuning approach, we conducted several cross-domain experiments. This evaluation is critical because real-world applications often involve deploying models on data distributions that differ from the training data. We compare our method’s performance against full fine-tuning and the established baselines under these domain shift conditions, with detailed results presented in Table 6.4.

The core idea of this evaluation is to train the models on a dataset from one specific domain and then evaluate their performance on a related but distinct dataset from a different domain. This setup directly tests how well the learned adaptations transfer when faced with variations in style, vocabulary, length, or data source characteristics, while the underlying task remains similar.

We selected specific pairs of datasets for this purpose:

- **Sentiment Analysis:** Stanford Sentiment Treebank (SST-2) Dataset [38] and IMDb Movie Reviews Dataset [29]. Both datasets deal with movie review sentiment, but they differ significantly. SST-2 typically involves shorter phrases or sentences, while IMDb consists of longer, paragraph-length reviews. Training on SST-2 and testing on IMDb (and vice-versa) evaluates robustness to text length and stylistic variations in sentiment expression.
- **Harmful Content Detection:** Offensive Language Identification Dataset (OLID) [49] and Dynamically Generated Hate Speech Dataset (Dyn. Hate) [42]. Both tasks involve identifying toxic or offensive content. However, OLID is a standard benchmark often containing curated examples, whereas Dyn. Hate utilizes dynamic generation, creating more challenging or adversarial examples that models might not have encountered. This pairing tests the model’s resilience to different data generation processes and potentially more subtle or complex forms of harmful language.

The results in Table 6.4 reveal several key insights into the generalization capabilities of our approach. Our method, specifically using the high-scoring k layers selection, demonstrates remarkable stability. When trained on one domain (e.g., SST-2) and tested on another (e.g., IMDb), the performance remains very close (typically within $<1\%$) to that achieved by full fine-tuning in the cross-domain setting. This is particularly noteworthy given that our approach only fine-tunes 25% of the transformer layers. It suggests that the knowledge adapted in these few selected layers is highly transferable.

Importantly, the cross-domain setting highlights a significant advantage of our method over the compared baselines (LIFT, RECAP, SlimFit, LoRA, I/O Similarity, Random selection). While these baselines might perform reasonably well within-domain, they exhibit a considerably larger drop in performance when faced with a domain shift. Our approach consistently outperforms them, often by a substantial margin, in these cross-domain evaluations. This indicates that the layers selected by our [CLS] similarity criterion retain more generalizable, domain-agnostic information.

This strong OOD performance hence suggests that our training-free layer selection, based on inter-layer [CLS] token similarity, effectively identifies layers whose adaptation contributes meaningfully to the task without overfitting to the specific cues of the source domain. Because the selection relies on the intrinsic representational structure revealed by [CLS] token similarities before task-specific fine-tuning, it might be less susceptible

Table 6.4: Comparison of Cross-Domain Performance. For our method, we show the results for the high scoring k layers, for $k=3$. **Bold** indicates the best-performing model and underline indicates the second best.

Method	Training Set	Evaluation Set	Accuracy
Full Fine-tuning	SST-2	IMDB	88.8
Random Selection			84.5
LoRA			85.3
LIFT			84.8
RECAP			85.0
SlimFit			<u>86.2</u>
I/O Similarity			85.8
Our Method			87.6
Full Fine-tuning	IMDB	SST-2	89.0
Random Selection			84.2
LoRA			86.9
LIFT			84.7
RECAP			85.1
SlimFit			<u>87.0</u>
I/O Similarity			86.5
Our Method			88.5
Full Fine-tuning	OLID	Dyn. Hate	52.8
Random Selection			46.4
LoRA			47.8
LIFT			46.7
RECAP			48.2
SlimFit			<u>51.3</u>
I/O Similarity			47.6
Our Method			53.0
Full Fine-tuning	Dyn. Hate	OLID	71.7
Random Selection			66.0
LoRA			68.6
LIFT			66.5
RECAP			67.3
SlimFit			<u>69.2</u>
I/O Similarity			68.8
Our Method			71.6

to capturing domain-specific artifacts compared to methods that rely heavily on training dynamics or gradients within the source domain. The preservation of frozen layers, representing the bulk of the pre-trained knowledge, also contributes to this robustness.

The ability to maintain high performance across domains, even with significantly reduced parameter tuning, underscores the practical utility of our approach. Models fine-tuned using our method are likely to be more reliable when deployed in dynamic environments where data characteristics may evolve or differ from the initial training set.

As such, the cross-domain evaluations strongly indicate that our proposed method not only achieves parameter and computational efficiency but also enhances the model’s generalization capabilities. The approach effectively balances adaptation to the target task with the preservation of core, transferable knowledge, leading to improved robustness against domain shifts compared to both full fine-tuning and existing partial fine-tuning techniques. While we presented results for the high-scoring k layer strategy here, similar trends of improved robustness over baselines were observed for our other selection variants (low-scoring, block-wise) as well. This further highlights the effectiveness of leveraging inter-layer representational similarity for robust and efficient fine-tuning.

6.3 Ablation

To better understand the behavior and sensitivity of our proposed training-free layer selection method, we conduct several ablation studies. These analyses explore the impact of key hyperparameters and the method’s applicability across different model architectures, providing deeper insights into its performance characteristics.

6.3.1 Performance across Different k Values

A critical hyperparameter in our approach is k , the number of layers selected for fine-tuning. Selecting an appropriate k involves balancing performance gains against computational savings (parameter count and training speedup). To investigate this trade-off empirically, we evaluated our selection strategies (High Scoring, Low Scoring, Block-wise Highest, Block-wise Lowest) across a range of k values from 1 to 11 (out of 12 total layers) using the BERT_{BASE} model. The results, including accuracy and speedup factor relative to full fine-tuning, are illustrated for the SST-2 (Figure 4) and QNLI (Figure 5) datasets.

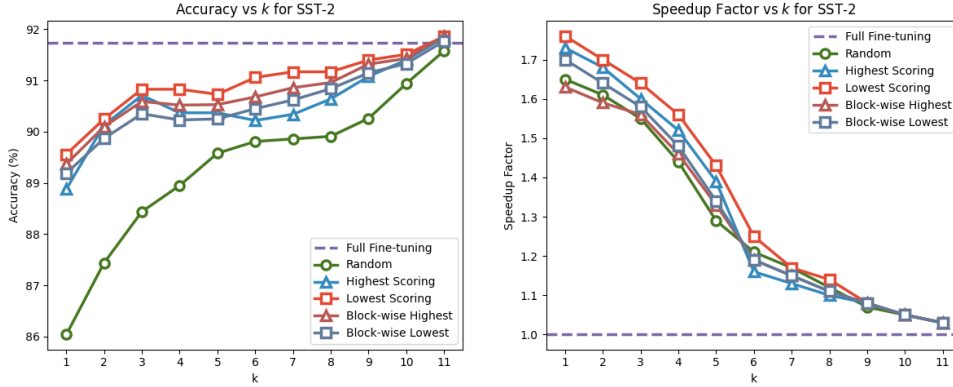


Figure 4: Comparison of accuracy and speedup across k for the SST-2 dataset

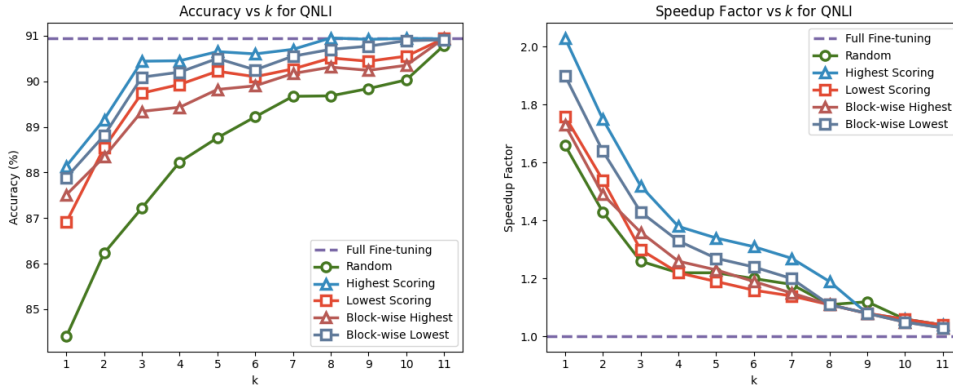


Figure 5: Comparison of accuracy and speedup across k for the QNLI dataset

As observed in Figures 4 and 5, accuracy generally increases as k grows, which is expected since more parameters are being updated. However, the rate of improvement typically diminishes significantly after the initial few layers are included (e.g., $k = 3$ or $k = 4$). For SST-2 (Figure 4), fine-tuning the lowest-scoring layers consistently yields slightly better results than the highest-scoring ones across most k values. Conversely, for QNLI (Figure 5), fine-tuning the highest-scoring layers tends to outperform fine-tuning

the lowest-scoring layers, particularly for smaller k . This suggests that the optimal strategy (selecting high vs. low similarity layers) might be task-dependent, likely related to whether the task benefits more from refining generic representations (high similarity) or adapting unique, task-specific features (low similarity). The block-wise strategies often demonstrate strong performance, sometimes exceeding the simple high/low scoring approaches, indicating the benefit of distributing selected layers across the model’s depth.

As for speedup gains compared to full fine-tuning, the training speedup factor exhibits a clear inverse relationship with k . As more layers are selected for fine-tuning (k increases), the speedup compared to full fine-tuning decreases. This decrease is often more pronounced for smaller values of k and becomes more gradual as k approaches the total number of layers L . For instance, moving from $k = 1$ to $k = 3$ results in a noticeable drop in speedup, but the subsequent drops for increasing k become progressively smaller.

Based on these observations across multiple datasets (SST-2 and QNLI shown, others analyzed internally), we identified $k = 3$ as offering a compelling balance between performance and efficiency for the 12-layer BERT_{BASE} model. At $k = 3$, our methods achieve accuracy close to that obtained with higher k values (often within 1-2% of the maximum partial fine-tuning accuracy) while retaining a significant speedup (around 1.5x) and reducing trainable parameters by 75%. Increasing k beyond 3 generally yields diminishing returns in accuracy at the cost of reduced speedup and increased parameter count. Therefore, $k = 3$ was chosen as the default value for our main experiments presented in Tables 6.1 and 6.2 to provide a consistent and efficient comparison point.

6.3.2 Performance across Different LLMs

To assess the generalizability of our training-free selection approach beyond BERT_{BASE}, we evaluated its performance on a diverse set of pre-trained language models with varying architectures and sizes. This included RoBERTa_{BASE}, DeBERTa_{BASE}, ELECTRA_{BASE}, as well as larger and more recent models like Gemma-2-2B and TinyLlama-1.1B (described in Chapter 4). For these experiments, we maintained $k = 3$ (adjusted proportionally for models with different layer counts if necessary) and compared our High Scoring and Low Scoring strategies against simpler heuristic baselines: random k layer selection, bottom k layer selection, and top k layer selection. The results are summarized in Table 6.5 across four representative tasks (RTE, MRPC, SST-2, STS-B).

Table 6.5 demonstrates that our [CLS] token similarity-based selection method consistently outperforms the heuristic baselines (random, top-k, bottom-k) across all tested models. For example, on RoBERTa, our High Scoring approach achieves an average score of 84.4, surpassing the best baseline (Top k layers) score of 82.4. Similarly, on DeBERTa, our Low Scoring strategy reaches 88.2 on average, compared to 86.5 for the best baseline (Bottom k layers). This consistency highlights that the underlying principle, using inter-layer [CLS] similarity to identify relevant layers, is not specific to BERT’s architecture but translates effectively to other Transformer-based models.

Interestingly, similar to the observation with different tasks, the best performing strategy between High Scoring and Low Scoring layers can vary depending on the specific model architecture. For instance, High Scoring performed best for RoBERTa, Gemma, and TinyLlama, while Low Scoring was superior for DeBERTa and ELECTRA in our experiments. This suggests potential interactions between the model’s pre-training objective, its internal representation dynamics, and which layers (generic vs. specialized) are most beneficial to adapt via our method.

Finally, the approach remains effective even for the larger and compact models like Gemma-2-2B and TinyLlama-1.1B. Our selection strategies significantly outperform the

baselines on these models too (e.g., 88.5 vs 87.7 for Gemma, 81.7 vs 80.8 for TinyLlama). This demonstrates the broad applicability of the method, extending beyond base models to more compact architectures often used in resource-constrained environments.

Table 6.5: Comparison of performance for different language models. We use $k=3$ for all layer selection methods.

Model	Method	RTE (acc)	MRPC (F1)	SST-2 (acc)	STS-B (Pear.)	Avg.
RoBERTa	Random k layers	67.4	82.1	90.2	84.8	81.1
	Bottom k layers	68.3	82.9	91.4	85.3	82.0
	Top k layers	69.1	83.6	90.8	86.2	82.4
	High Scoring k layers	71.8	85.1	92.2	88.4	84.4
	Low Scoring k layers	72.9	84.6	92.0	87.3	<u>84.2</u>
DeBERTa	Random k layers	78.4	84.8	92.8	87.2	85.8
	Bottom k layers	79.4	85.2	93.4	88.1	86.5
	Top k layers	79.7	85.3	93.2	87.0	86.3
	High Scoring k layers	81.2	87.4	93.2	89.7	<u>87.9</u>
	Low Scoring k layers	83.4	86.2	94.6	88.5	88.2
ELECTRA	Random k layers	71.3	82.6	88.4	83.8	81.5
	Bottom k layers	72.0	83.8	87.3	85.6	82.2
	Top k layers	72.6	83.9	89.2	84.2	82.5
	High Scoring k layers	74.7	86.3	88.5	87.6	<u>84.3</u>
	Low Scoring k layers	74.0	85.8	93.2	89.1	85.5
Gemma-2-2B	Random k layers	74.4	90.2	95.8	90.3	<u>87.7</u>
	Bottom k layers	69.0	85.8	93.6	82.3	82.7
	Top k layers	75.8	82.3	90.0	85.8	83.5
	High Scoring k layers	78.9	89.8	96.8	88.6	88.5
	Low Scoring k layers	75.4	89.1	95.3	87.2	86.8
TinyLlama-1.1B	Random k layers	61.0	84.9	93.6	83.5	<u>80.8</u>
	Bottom k layers	59.9	81.5	56.4	84.9	70.7
	Top k layers	61.7	81.7	92.7	68.7	76.2
	High Scoring k layers	63.8	86.4	92.4	84.3	81.7
	Low Scoring k layers	62.5	85.7	86.2	86.7	80.3

7 Conclusion and Future Work

7.1 Conclusion

This work confronted the escalating challenge of adapting ever-larger pre-trained language models, proposing and validating a distinctly efficient paradigm for partial fine-tuning. We introduced a novel layer selection strategy predicated entirely on the geometric relationships between [CLS] token representations across layers, computed via cosine similarity during a single, training-free forward pass. This approach fundamentally departs from methods reliant on computationally intensive gradient analysis, dynamic updates, or purely heuristic positional assumptions. Our extensive empirical investigation, spanning fifteen diverse NLP benchmarks, unequivocally established the efficacy of this strategy. The method not only consistently surpassed a wide array of contemporary baselines, ranging from simple heuristics to sophisticated dynamic selection techniques like LIFT, RECAP, and SlimFit, as well as I/O similarity approaches, but also achieved downstream task performance remarkably proximate to full fine-tuning, often closing the gap to within 1-2%, while dramatically reducing trainable parameters by roughly 75% and accelerating training times significantly.

The success of leveraging inter-layer [CLS] token similarity offers a compelling insight into the nature of pre-trained representations and the fine-tuning process itself. It suggests that the inherent structural and semantic hierarchy learned during pre-training leaves a discernible signature in the relative alignment of these summary ([CLS]) vectors across layers. This static, pre-computed signal effectively identifies layers acting as crucial hubs for information integration (high similarity) or critical points of representational transformation (low similarity). The ability of this training-free signal to predict layers whose adaptation yields high downstream performance implies that much of the necessary groundwork for task-specific adjustment is already encoded within the model’s architecture and pre-trained state, reducing the perceived complexity of the fine-tuning challenge. It hints that effective adaptation might be less about uniformly adjusting all parameters and more about strategically perturbing specific, structurally pivotal connections in the network’s information flow.

Furthermore, the pronounced robustness of our method in cross-domain evaluations is particularly noteworthy. While many partial fine-tuning techniques exhibited significant performance degradation when faced with domain shifts, our approach demonstrated remarkable stability, often maintaining performance parity with full fine-tuning in these challenging OOD scenarios. This resilience suggests that by grounding layer selection in the stable, pre-existing representational geometry rather than transient training dynamics (which might capture source-domain artifacts), our method preferentially adapts layers involved in more fundamental, domain-agnostic linguistic processing. Freezing the majority of the network inherently preserves a vast store of general knowledge, while the [CLS]-similarity criterion guides the adaptation towards layers whose adjustments are broadly beneficial, rather than narrowly optimizing for source-domain statistics. This

characteristic significantly enhances the practical utility of the method, promising greater reliability when deploying models in real-world, dynamic environments.

The observed variability in the optimal selection strategy (High Scoring, Low Scoring, Block-wise) across different tasks and even different model architectures provides another layer of insight. It underscores that there is no single “most important” set of layers universally applicable across all scenarios. Instead, the ideal adaptation targets likely depend on a complex interplay between the specific demands of the downstream task (e.g., requiring refinement of broad semantics vs. emphasis on unique syntactic cues) and the internal representational landscape shaped by the model’s unique pre-training conditions. The success of the block-wise strategies, in particular, highlights the value of distributing adaptation across different levels of representational abstraction, preventing over-specialization at any single depth and promoting a more comprehensive model adjustment.

Ultimately, this work delivers more than just an efficient algorithm; it contributes to a more fundamental understanding of transformer adaptation. By demonstrating that a simple, static measure of inter-layer representational similarity can guide highly effective and robust partial fine-tuning, we challenge the necessity for complex, dynamic selection mechanisms in many cases. The findings suggest a path towards more computationally frugal, accessible, and reliable fine-tuning methodologies, grounded in a deeper appreciation of the information encoded within the structure of pre-trained models even before task-specific training commences. This training-free selection principle offers a potent tool for democratizing the use of large language models and encourages further exploration into how pre-trained network structure informs downstream task adaptability.

7.2 Limitations

While this study demonstrates the effectiveness and efficiency of the proposed layer selection method across a diverse set of tasks and several model architectures, its scope has certain boundaries. The primary evaluation focused extensively on BERT-base, with ablations covering other models; however, performance dynamics on a wider range of architectures, particularly extremely large-scale models or those with significantly different pre-training objectives (e.g., purely generative models), were not exhaustively explored due to computational constraints. Furthermore, the experiments were conducted exclusively on English language datasets, leaving the method’s applicability and effectiveness for multilingual or low-resource language scenarios as an open question requiring further investigation. The task types were also primarily centered around discriminative tasks like classification and regression, and the performance characteristics for sequence generation or other complex NLP tasks using this partial fine-tuning approach remain to be characterized.

Additionally, the selection of the hyperparameter k , representing the number of layers to fine-tune, was standardized to $k=3$ for main comparisons based on empirical observations balancing performance and efficiency. Although this value proved effective across many scenarios, the optimal k may vary depending on the specific model, dataset complexity, and task requirements. Determining the ideal k for a novel application might necessitate preliminary exploration, adding a step not required by methods that adapt all layers uniformly, albeit less efficiently.

While the empirical success across tested models supports the underlying principle, a deeper theoretical investigation into precisely why the [CLS] token similarity serves as such a robust indicator across diverse architectural variations could further strengthen

the foundational understanding, though it is beyond the scope of the current empirical focus.

7.3 Impact on the Society

7.3.1 Sustainability of the Work

The sustainability of this research primarily stems from its contribution to resource efficiency in AI development. By significantly reducing the number of trainable parameters and accelerating training times compared to full fine-tuning, while maintaining competitive performance, the proposed method lowers the computational resources required to adapt LLMs for specific tasks. This reduction translates directly into lower energy consumption during the fine-tuning phase, contributing to more environmentally sustainable AI practices.

Moreover, the training-free nature of the layer selection process itself simplifies the adaptation pipeline, reducing the overhead associated with more complex dynamic or gradient-based selection methods. This methodological simplicity enhances the reproducibility and potential for wider adoption of efficient fine-tuning strategies. The demonstrated robustness, particularly in cross-domain scenarios, also contributes to sustainability by reducing the need for frequent re-tuning when models encounter slightly different data distributions in real-world deployment, leading to longer effective lifecycles for fine-tuned models. This work provides a foundation for future research into computationally frugal AI, promoting a more sustainable trajectory for the field.

7.3.2 Social and Environmental Effects and Analysis

The primary environmental benefit of this work lies in the reduced energy consumption and associated carbon footprint during the model adaptation phase. While the pre-training of large models remains resource-intensive, optimizing the downstream fine-tuning process is a critical step towards mitigating the overall environmental impact of deploying LLMs widely. By enabling effective adaptation with significantly fewer computational resources, our method makes progress in this direction.

Socially, the increased efficiency has the potential to democratize access to powerful AI capabilities. Full fine-tuning often requires substantial computational infrastructure, limiting its feasibility for smaller organizations, academic researchers, non-profits, or developers in resource-constrained environments. By lowering these barriers, our method could foster broader innovation and application of LLM technologies across diverse sectors and communities. The improved cross-domain robustness could also lead to more reliable AI tools that perform consistently across varied contexts or user groups.

However, it is crucial to analyze the potential negative effects. This method operates on pre-trained models, which inevitably inherit biases present in their vast training data. While our selection technique itself is based on structural properties ([CLS] similarity) and is unlikely to introduce new systemic biases, it does not inherently mitigate the biases already present in the base model. Making adaptation more efficient could inadvertently accelerate the deployment of potentially biased or unfair AI systems if not paired with rigorous evaluation and mitigation strategies.

7.3.3 Ethical Considerations

Several ethical considerations arise from this work. Of particular importance, as mentioned, is the issue of bias and fairness. The efficiency gains offered by our method do not absolve developers of the responsibility to address biases encoded within the pre-trained LLMs. The layer selection process, while effective for performance, is not designed for bias detection or mitigation. Therefore, practitioners using this method must remain vigilant, employing appropriate datasets for fine-tuning, conducting thorough bias audits of the fine-tuned models, and implementing fairness-aware techniques where necessary.

Another consideration is transparency. While the layer selection mechanism itself is relatively straightforward and interpretable compared to complex dynamic methods, the overall behavior of the resulting fine-tuned LLM remains complex and often opaque, characteristic of deep learning models. Accountability also remains paramount; the developers and deployers of models fine-tuned using this technique are responsible for their behavior and impact. The increased ease and reduced cost of adaptation could potentially lower the threshold for misuse. Malicious actors might find it easier to repurpose LLMs for generating misinformation, spam, or other harmful content. While this is a general concern for powerful AI, increased efficiency necessitates heightened awareness and proactive measures against potential dual-use scenarios.

Finally, data privacy remains relevant during the fine-tuning stage, as task-specific datasets may contain sensitive information, requiring adherence to standard data protection practices. Ultimately, the ethical application of this efficient fine-tuning method hinges on responsible development practices and a commitment to ongoing evaluation and mitigation of potential harms.

7.4 Future Work

Future research directions building upon this work could include investigating the scalability of the [CLS] similarity-based layer selection to vastly larger language models (e.g., >100B parameters) and explore its efficacy within multilingual or cross-lingual fine-tuning scenarios, potentially adapting the similarity computation for different linguistic structures.

Extending the methodology to assess its suitability for generative tasks, such as text summarization or machine translation, where the relevance of aggregate [CLS] representations might differ, as well as vision tasks, such as image classification or segmentation using Vision Transformers (ViTs), presents other valuable avenues.

Furthermore, developing adaptive or automated techniques for determining the optimal number of layers (k) to fine-tune based on task characteristics or initial score distributions, moving beyond fixed empirical choices, could enhance practical applicability.

A deeper theoretical investigation into the representational geometry and information flow captured by inter-layer [CLS] similarity is also warranted to formalize why this static, training-free metric effectively identifies layers crucial for adaptation across diverse architectures.

Finally, exploring potential combinations by integrating this layer selection strategy with other parameter-efficient fine-tuning methods, for instance, applying low-rank updates exclusively to the selected layers, could unlock further optimizations in the performance-efficiency landscape.

Bibliography

- [1] ARDAKANI, A., HAAN, A., TAN, S., POPOVICI, D. T., CHEUNG, A., IANCU, C., AND SEN, K. SlimFit: Memory-efficient fine-tuning of transformer-based models using training dynamics. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)* (2024), pp. 6218–6236.
- [2] ARORA, S., MAY, A., ZHANG, J., AND RÉ, C. Contextual embeddings: When are they worth it? In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)* (2020), pp. 2650–2663.
- [3] ASHKBOOS, S., CROCI, M. L., DO NASCIMENTO, M. G., HOEFLER, T., AND HENSMAN, J. SliceGPT: Compress large language models by deleting rows and columns. In *The Twelfth International Conference on Learning Representations (ICLR)* (2024).
- [4] BOWMAN, S. R., ANGELI, G., POTTS, C., AND MANNING, C. D. A large annotated corpus for learning natural language inference. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (2015), pp. 632–642.
- [5] CHARPENTIER, L. G. G., AND SAMUEL, D. Not all layers are equally as important: Every layer counts BERT. In *Proceedings of the BabyLM Challenge at the 27th Conference on Computational Natural Language Learning* (2023), pp. 238–252.
- [6] CHEN, T., KORNBLITH, S., NOROUZI, M., AND HINTON, G. A simple framework for contrastive learning of visual representations. In *International Conference on Machine Learning (ICML)* (2020), pp. 1597–1607.
- [7] CHEN, X., HU, Y., ZHANG, J., WANG, Y., LI, C., AND CHEN, H. Streamlining redundant layers to compress large language models. In *The Thirteenth International Conference on Learning Representations (ICLR)* (2025).
- [8] CHEN, X., HU, Y., ZHANG, J., WANG, Y., LI, C., AND CHEN, H. Tracing representation progression: Analyzing and enhancing layer-wise similarity. In *The Thirteenth International Conference on Learning Representations (ICLR)* (2025).
- [9] CHOI, H., KIM, J., JOE, S., AND GWON, Y. Evaluation of bert and albert sentence embedding performance on downstream nlp tasks. In *25th International Conference on Pattern Recognition (ICPR)* (2021), pp. 5482–5487.
- [10] CLARK, K., LUONG, M.-T., LE, Q. V., AND MANNING, C. D. Electra: Pre-training text encoders as discriminators rather than generators. In *International Conference on Learning Representations (ICLR)* (2020).

- [11] DEVLIN, J., CHANG, M.-W., LEE, K., AND TOUTANOVA, K. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)* (2019), pp. 4171–4186.
- [12] DING, M., YANG, Z., HONG, W., ZHENG, W., ZHOU, C., YIN, D., LIN, J., ZOU, X., SHAO, Z., YANG, H., AND TANG, J. Cogview: Mastering text-to-image generation via transformers. In *Advances in Neural Information Processing Systems (NeurIPS)* (2021), vol. 34, pp. 19822–19835.
- [13] GAO, T., FISCH, A., AND CHEN, D. Making pre-trained language models better few-shot learners. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing* (2021), pp. 3816–3830.
- [14] GU, J., ALETI, A., CHEN, C., AND ZHANG, H. A semantic-based layer freezing approach to efficient fine-tuning of language models. *arXiv preprint arXiv:2406.11753* (2024).
- [15] HE, P., LIU, X., GAO, J., AND CHEN, W. DeBERTa: Decoding-enhanced bert with disentangled attention. In *International Conference on Learning Representations (ICLR)* (2021).
- [16] HE, S., SUN, G., SHEN, Z., AND LI, A. What matters in transformers? not all attention is needed. *arXiv preprint arXiv:2406.15786* (2024).
- [17] HOSSEINI, M., MUNIA, M., AND KHAN, L. BERT has more to offer: BERT layers combination yields better sentence embeddings. In *Findings of the Association for Computational Linguistics: EMNLP 2023* (2023), pp. 15419–15431.
- [18] HOULSBY, N., GIURGIU, A., JASTRZEBSKI, S., MORRONE, B., DE LAROUSSILHE, Q., GESMUNDO, A., ATTARIYAN, M., AND GELLY, S. Parameter-efficient transfer learning for nlp. In *International Conference on Machine Learning (ICML)* (2019), pp. 2790–2799.
- [19] HU, E. J., SHEN, Y., WALLIS, P., ALLEN-ZHU, Z., LI, Y., WANG, S., WANG, L., AND CHEN, W. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)* (2022).
- [20] HU, Y., ZHANG, J., ZHAO, Z., ZHAO, C., CHEN, X., LI, C., AND CHEN, H. sp³: Enhancing structured pruning via PCA projection. In *Findings of the Association for Computational Linguistics: ACL 2024* (2024), pp. 3150–3170.
- [21] ILHAN, F., SU, G., TEKIN, S. F., HUANG, T., HU, S., AND LIU, L. Resource-efficient transformer pruning for finetuning of large models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2024), pp. 16206–16215.
- [22] INGLE, D., TRIPATHI, R. K., KUMAR, A., PATEL, K., AND VEPA, J. Investigating the characteristics of a transformer in a few-shot setup: Does freezing layers in RoBERTa help? In *Proceedings of the Fifth BlackboxNLP Workshop on Analyzing and Interpreting Neural Networks for NLP* (2022), pp. 238–248.

- [23] JAWAHAR, G., SAGOT, B., AND SEDDAH, D. What does BERT learn about the structure of language? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)* (2019), pp. 3651–3657.
- [24] LEE, J., TANG, R., AND LIN, J. What would elsa do? freezing layers during transformer fine-tuning. *arXiv preprint arXiv:1911.03090* (2019).
- [25] LESTER, B., AL-RFOU, R., AND CONSTANT, N. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (2021), pp. 3045–3059.
- [26] LI, X. L., AND LIANG, P. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)* (2021), pp. 4582–4597.
- [27] LIU, Y., OTT, M., GOYAL, N., DU, J., JOSHI, M., CHEN, D., LEVY, O., LEWIS, M., ZETTLEMOYER, L., AND STOYANOV, V. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* (2019).
- [28] LIU, Z., WANG, J., DAO, T., ZHOU, T., YUAN, B., SONG, Z., SHRIVASTAVA, A., ZHANG, C., TIAN, Y., RE, C., AND CHEN, B. Deja vu: Contextual sparsity for efficient llms at inference time. In *International Conference on Machine Learning (ICML)* (2023), pp. 22137–22176.
- [29] MAAS, A. L., DALY, R. E., PHAM, P. T., HUANG, D., NG, A. Y., AND POTTS, C. Learning word vectors for sentiment analysis. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies* (2011), pp. 142–150.
- [30] MCCARLEY, J. S., CHAKRAVARTI, R., AND SIL, A. Structured pruning of a bert-based question answering model. *arXiv preprint arXiv:1910.06360* (2019).
- [31] MICHEL, P., LEVY, O., AND NEUBIG, G. Are sixteen heads really better than one? In *Advances in Neural Information Processing Systems (NeurIPS)* (2019), vol. 32.
- [32] NIU, J., LU, W., AND PENN, G. Does BERT rediscover a classical NLP pipeline? In *Proceedings of the 29th International Conference on Computational Linguistics* (2022), pp. 3143–3153.
- [33] PAN, R., LIU, X., DIAO, S., PI, R., ZHANG, J., HAN, C., AND ZHANG, T. LISA: Layerwise importance sampling for memory-efficient large language model fine-tuning. *arXiv preprint arXiv:2403.17919* (2024).
- [34] PRASANNA, S., ROGERS, A., AND RUMSHISKY, A. When BERT Plays the Lottery, All Tickets Are Winning. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (2020), pp. 3208–3229.
- [35] REIMERS, N., AND GUREVYCH, I. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)* (2019), pp. 3982–3992.

- [36] SHEN, Z., LIU, Z., QIN, J., SAVVIDES, M., AND CHENG, K.-T. Partial is better than all: Revisiting fine-tuning strategy for few-shot learning. In *Proceedings of the AAAI conference on artificial intelligence* (2021), pp. 9594–9602.
- [37] SIMOULIN, A., PARK, N., LIU, X., AND YANG, G. Memory-efficient selective fine-tuning. In *Workshop on Efficient Systems for Foundation Models @ ICML2023* (2023).
- [38] SOCHER, R., ALEX PERELYGIN, J. W., CHUANG, J., MANNING, C. D., NG, A., AND POTTS, C. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (2013), pp. 1631–1642.
- [39] TEAM, G., RIVIERE, M., PATHAK, S., SESSA, P. G., HARDIN, C., BHUPATIRAJU, S., HUSSENOT, L., MESNARD, T., SHAHRIARI, B., RAMÉ, A., ET AL. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118* (2024).
- [40] VAN DER OUDERAA, T. F. A., NAGEL, M., BAALEN, M. V., AND BLANKEVOORT, T. The LLM surgeon. In *The Twelfth International Conference on Learning Representations (ICLR)* (2024).
- [41] VASWANI, A., SHAZEER, N., PARMAR, N., USZKOREIT, J., JONES, L., GOMEZ, A. N., KAISER, L., AND POLOSUKHIN, I. Attention is all you need. *Advances in neural information processing systems 30* (2017).
- [42] VIDGEN, B., THRUSH, T., WASEEM, Z., AND KIELA, D. Learning from the worst: Dynamically generated datasets to improve online hate detection. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)* (2021), pp. 1667–1682.
- [43] VOITA, E., TALBOT, D., MOISEEV, F., SENNRICH, R., AND TITOV, I. Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics* (2019), pp. 5797–5808.
- [44] VUCETIC, D., TAYARANIAN, M., ZIAEEFARD, M., CLARK, J. J., MEYER, B. H., AND GROSS, W. J. Efficient fine-tuning of bert models on the edge. In *2022 IEEE International Symposium on Circuits and Systems (ISCAS)* (2022), pp. 1838–1842.
- [45] WANG, A., SINGH, A., MICHAEL, J., HILL, F., LEVY, O., AND BOWMAN, S. R. Glue: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP* (2018), pp. 353–355.
- [46] XIA, M., GAO, T., ZENG, Z., AND CHEN, D. Sheared LLaMA: Accelerating language model pre-training via structured pruning. In *The Twelfth International Conference on Learning Representations (ICLR)* (2024).
- [47] XIONG, R., YANG, Y., HE, D., ZHENG, K., ZHENG, S., XING, C., ZHANG, H., LAN, Y., WANG, L., AND LIU, T.-Y. On layer normalization in the transformer architecture. In *International Conference on Machine Learning (ICML)* (2020), pp. 10524–10533.

- [48] ZAKEN, E. B., RAVFOGEL, S., AND GOLDBERG, Y. BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)* (2022), pp. 1–9.
- [49] ZAMPIERI, M., MALMASI, S., NAKOV, P., ROSENTHAL, S., FARRA, N., AND KUMAR, R. Predicting the type and target of offensive posts in social media. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)* (2019), pp. 1415–1420.
- [50] ZHANG, J.-G., HASHIMOTO, K., LIU, W., WU, C.-S., WAN, Y., YU, P. S., SOCHER, R., AND XIONG, C. Discriminative nearest neighbor few-shot intent detection by transferring natural language inference. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (2020), pp. 5064–5082.
- [51] ZHANG, P., ZENG, G., WANG, T., AND LU, W. Tinyllama: An open-source small language model. *arXiv preprint arXiv:2401.02385* (2024).
- [52] ZHU, L., HU, L., LIN, J., AND HAN, S. LIFT: Efficient layer-wise fine-tuning for large model models, 2024.